CS412 Affective Computing · Group 1

Code Walkthrough

Bisaya Post-Class Speech Affect Recognition

Line-by-line explanation of `bisaya_audio_ser_group1.ipynb`



Covers all 25 code cells (821 lines) of the final audio-only notebook.
Expected outputs are from your own run of the notebook on the 30 recordings.
October 2026

Contents

How to read this guide 3

The pipeline at a glance 3

Notebook map 3

Libraries used 4

1 Installing the packages 6

2 Importing the libraries 6

3 Setup: folders and settings 8

4 Loading the metadata and making the labels 10

5 Checking the metadata against the protocol 12

6 Matching files to the metadata 15

7 Exploratory analysis of the ratings (RQ2) 17

8 RQ1: how students felt after class 19

9 Audio quality report 21

10 Preprocessing and feature extraction 23

11 Demo: one recording to one feature vector 29

12 Labelled spectrogram figures 32

13 Waveforms and clearer spectrograms 34

14 RQ3: do the features differ between classes? 38

15 Spectrogram embeddings with ResNet18 41

16 Saving the dataset 42

17 Training and testing the models (RQ4, RQ5) 44

18 Confusion matrices and per-class results 47

19 Is the accuracy really above the baseline? 49

20 Sensitivity check without the two Libog recordings 51

21 Permutation test 52

22 Same word, different feeling 54

23 Prototype 57

A Output files 59

B Variables reference **60**

C Settings at a glance **61**

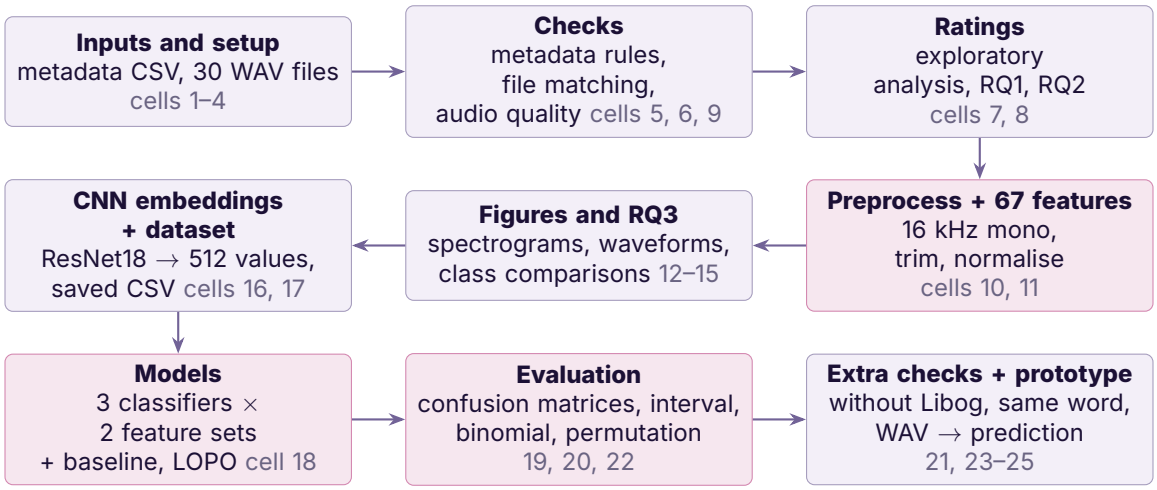
D Glossary **62**

How to read this guide

This guide follows the notebook `bisaya_audio_ser_group1.ipynb` from top to bottom. Each numbered section covers one code cell, or two cells that belong together, and explains every line in it.

- **Code is shown exactly as it is in the notebook**, split into short pieces. After each piece comes the explanation of those lines.
- **The numbers on the left are line numbers inside that cell**, counting from 1 at the top of the cell. Turn on line numbers in Jupyter (View menu → Show Line Numbers) and they will match what you see.
- **"Code cell *k*"** counts only the code cells, from the top. After a clean run from top to bottom (including the `%pip` cell), it is also the number Jupyter shows in the brackets, `In [k]`.
- Each section starts with a box listing what the cell **uses** (variables made earlier), what it **creates** (new variables), and what it **saves** to `RESULTSV3/`.
- **Expected output** boxes show what the cell printed for your 30 recordings, so you can check your run and quote the numbers.
- **Note** boxes point out cautions and weak spots in the method. They are the things a panelist is most likely to ask about.

The pipeline at a glance



The highlighted boxes are the core of the method: turning each recording into numbers, training the models, and testing them on people they never saw.

Notebook map

Code cell	Notebook heading	What it does	Section
1	(top)	Installs the Python packages	1
2	(top)	Imports libraries, fixes the random seed	2
3	Setup	Folders, settings, word list, placeholders	3
4	Metadata and labels	Loads the CSV, makes the binary labels	4

Code cell	Notebook heading	What it does	Section
5	Checking the metadata	Checks filenames and columns against the protocol	5
6	Do the files match the metadata?	Matches each row to its WAV and MP4	6
7	Exploratory analysis	Ratings by activity, session and word (RQ2)	7
8	RQ1	Self-reported states, scatter plot, heatmap	8
9	Audio quality	Length, level, clipping and silence of each WAV	9
10	Feature extraction	Preprocessing, 67 features, CNN images	10
11	Example: one recording ...	One WAV turned into its feature vector (demo)	11
12	(no heading)	Labelled spectrogram figures	12
13	Waveforms and clearer spectrograms	Waveform + pitch figures, word galleries	13
14–15	RQ3	Feature differences between classes	14
16	Spectrogram embeddings	ResNet18 turns each image into 512 numbers	15
17	Saving the dataset	Writes the 30 × 78 dataset and dictionary	16
18	Models (RQ4, RQ5)	Six models + baseline, leave-one-participant-out	17
19	Confusion matrices	Best model per target, per-class metrics	18
20	Is the accuracy really above the baseline?	Wilson interval and binomial test	19
21	Without the two Libog recordings	Reruns the top models on 28 recordings	20
22	Permutation test	1000 label shuffles → p-value	21
23–24	Same word, different feeling	Holds the spoken word constant	22
25	Prototype	Predicts affect for one WAV file	23

Where each part of the live demo is

Demo part	Code cells	What to point at
A. Raw data and dataset verification	4, 5, 6	30 recordings from 30 participants; no metadata problems; every row has its WAV and MP4; all labels valid (18/12 and 20/10)
B. Preprocessing and feature extraction	10, 11 (13 for figures)	The four-step table for Y1-014, the 67 feature names and values, and the final `True`
C. Machine learning and results	18, 19, 20, 22 (21, 24 if asked)	30 folds, the model comparison table, the two confusion matrices, the interval and p-values

Libraries used

Library	What the notebook uses it for
pandas (`pd`)	Tables ("DataFrames"): reading the CSV, grouping, counting, saving CSV files
NumPy (`np`)	Arrays of numbers and the maths on them
librosa	Loading and resampling audio, trimming silence, RMS, zero-crossing rate, spectral centroid, MFCCs, pitch (pYIN), mel-spectrograms
matplotlib (`plt`), seaborn (`sns`)	All figures; seaborn adds bar, strip, box, scatter and heatmap plots on top of matplotlib
Pillow (`PIL.Image`)	Saving and opening the spectrogram PNG images
PyTorch, torchvision	The pretrained ResNet18 network that turns each spectrogram image into 512 numbers

Library	What the notebook uses it for
scikit-learn (`sklearn`)	Scaling, PCA, the three classifiers, the baseline, leave-one-participant-out, metrics
SciPy (`scipy.stats`)	Mann–Whitney test, binomial test, Wilson confidence interval
Standard Python	`re` (filename patterns), `pathlib` (file paths), `math` (rounding up), `warnings` (hiding warnings)

1 Installing the packages

Code cell 1

The first code cell makes sure every library the notebook needs is installed in the Python environment that runs the notebook. It only has to work once per computer; after that it simply reports that everything is already installed.

Creates: nothing in Python; it installs packages on the computer.

```
1 %pip install librosa scikit-learn scipy matplotlib pillow torch torchvision seaborn
      ↪ pandas numpy
```

- **Line 1.** `%pip` is a Jupyter "magic" command: it runs the package installer `pip` inside the same Python environment as the notebook, so the packages end up where the notebook can import them. It installs **librosa** (audio), **scikit-learn** (models and evaluation), **scipy** (statistical tests), **matplotlib** and **seaborn** (figures), **pillow** (image files), **torch** and **torchvision** (the pretrained ResNet18), **pandas** (tables) and **numpy** (arrays).

Note. If a package is installed for the first time, restart the kernel (Kernel → Restart) before running the next cell, so Python picks up the new package. torch is a large download (several hundred MB).

2 Importing the libraries

Code cell 2

This cell loads every library and function the rest of the notebook uses, then fixes the random seed so results come out the same every time the notebook is run.

Creates: the short names `np`, `pd`, `plt`, `sns`, `T`, `models` and the imported functions; `SEED` = 42.

```
1 import math
2 import re
3 import warnings
4 from pathlib import Path
```

These four come with Python itself.

- **Line 1.** `math` gives `math.ceil` (round up), used in code cell 12 to work out how many rows of plots are needed.
- **Line 2.** `re` is Python's regular-expression module. Code cell 5 uses it to check that every filename follows the `Code_Year_Session_Word.wav` pattern.
- **Line 3.** `warnings` lets line 29 hide warning messages.
- **Line 4.** `Path` represents a file or folder path. Paths can be joined with `/` (for example `out_dir / "datasets"`), and the same code works on Windows, Mac and Linux.

```
6  import librosa
7  import librosa.display
8  import matplotlib
9  import matplotlib.pyplot as plt
10 import numpy as np
11 import pandas as pd
12 import seaborn as sns
13 import torch
14 import torchvision.models as models
15 import torchvision.transforms as T
16 from IPython.display import display
17 from PIL import Image
```

- **Line 6.** `librosa` is the audio library: loading files, resampling, trimming, and all the acoustic features.
- **Line 7.** `librosa.display` has to be imported separately; it provides `specshow`, which draws spectrograms with proper time and frequency axes.
- **Line 8.** `matplotlib` itself is needed for its colour maps (`matplotlib.colormaps["magma"]`), which colour the CNN input images in code cell 10.
- **Line 9.** `matplotlib.pyplot`, called `plt`, creates and saves every figure.
- **Line 10.** `numpy`, called `np`, handles arrays of numbers and the maths on them.
- **Line 11.** `pandas`, called `pd`, handles tables (DataFrames): reading CSV files, grouping, counting and saving.
- **Line 12.** `seaborn`, called `sns`, draws statistical plots (bar, strip, box, scatter, heatmap) on top of matplotlib.
- **Line 13.** `torch` is PyTorch, the deep-learning library that runs the ResNet18 network.
- **Line 14.** `torchvision.models` contains ready-made networks, including ResNet18 with its pre-trained weights.
- **Line 15.** `torchvision.transforms`, called `T`, converts an image into the tensor format the network expects.
- **Line 16.** `display` shows a DataFrame as a formatted table in the middle of a cell. `print` would show plain text, and only the last line of a cell is displayed automatically.
- **Line 17.** `Image` from Pillow opens, resizes and saves PNG images.

```
18 from scipy.stats import binomtest, mannwhitneyu
19 from sklearn.decomposition import PCA
20 from sklearn.dummy import DummyClassifier
21 from sklearn.ensemble import RandomForestClassifier
22 from sklearn.linear_model import LogisticRegression
23 from sklearn.metrics import accuracy_score, classification_report, confusion_matrix,
       ↪ f1_score
24 from sklearn.model_selection import LeaveOneGroupOut, cross_val_predict
25 from sklearn.pipeline import make_pipeline
26 from sklearn.preprocessing import StandardScaler
```

```
27 from sklearn.svm import SVC
```

- **Line 18.** From SciPy's statistics module: `binomtest` (binomial test and Wilson interval, code cell 20) and `mannwhitneyu` (the rank test for RQ3, code cell 14).
- **Line 19.** `PCA` (principal component analysis) reduces many features to a few combined ones.
- **Line 20.** `DummyClassifier` is the majority-class baseline: a "model" that ignores the audio.
- **Line 21.** `RandomForestClassifier`, one of the three real models.
- **Line 22.** `LogisticRegression`, another of the three models.
- **Line 23.** The scoring functions: `accuracy_score`, `classification_report` (precision, recall, F1 per class), `confusion_matrix`, `f1_score`.
- **Line 24.** `LeaveOneGroupOut` builds the leave-one-participant-out folds; `cross_val_predict` trains on each fold and collects the predictions for the left-out person.
- **Line 25.** `make_pipeline` chains steps (scaling, PCA, classifier) into one model that is trained as a unit.
- **Line 26.** `StandardScaler` rescales every feature to mean 0 and standard deviation 1.
- **Line 27.** `SVC` is the support vector machine, the third model.

```
29 warnings.filterwarnings("ignore")
30 sns.set_theme(style="whitegrid")
31 SEED = 42
32 np.random.seed(SEED)
```

- **Line 29.** Hides all warning messages so the output stays readable. The downside is that real warnings are hidden too; comment this line out when debugging.
- **Line 30.** Sets seaborn's look for all figures: white background with light grid lines.
- **Line 31.** `SEED = 42` is the fixed random seed used everywhere randomness appears (PCA, random forest, the jitter in the scatter plot, the permutation test). Any number would do; what matters is that it stays the same, so every run gives identical results.
- **Line 32.** Seeds NumPy's global random generator as a safety net, for any function that uses it without its own seed.

3 Setup: folders and settings

Code cell 3

Notebook heading: Setup

All the settings that might need changing are collected in this one cell: where the data is, where results go, the audio settings, the model settings and the protocol rules. Later cells refer to these names instead of repeating the numbers.

Creates: the folder paths, `SR`, `CLIP_SEC`, `N_MELS`, `IMG_SIZE`, `N_PCA`, `N_PERM`, `MIN_VALID`, `GROUP_YEAR`, `WORDS`, `session_cutoff`, `missing_reason`, and the helper function `save_fig`.
Saves to the results folder: nothing yet, but it creates the empty folders `RESULTSV3/`, `SPECTROGRAMS/`, `spectrograms_labelled/` and `datasets/`.

```
1  audio_dir = Path("audio")
2  video_dir = Path("video")
3  meta_csv = Path("GROUP 1_Metadata.csv")
```

- **Lines 1–3.** The three inputs. The paths are relative, so they point to `audio/`, `video/` and `GROUP 1_Metadata.csv` in the same folder as the notebook. The space in the CSV's name is fine inside `Path`.

```
5  out_dir = Path("RESULTSV3")
6  spec_dir = out_dir / "SPECTROGRAMS"                    # plain images for the CNN
7  spec_labelled_dir = out_dir / "spectrograms_labelled"  # same images with axes, for the
       ↪ report
8  data_dir = out_dir / "datasets"
9  for folder in [out_dir, spec_dir, spec_labelled_dir, data_dir]:
10     folder.mkdir(parents=True, exist_ok=True)
```

- **Line 5.** Every output goes under `RESULTSV3/`.
- **Line 6.** `SPECTROGRAMS/` holds the plain spectrogram images (no axes) that are fed to the CNN.
- **Line 7.** `spectrograms_labelled/` holds the same spectrograms drawn with axes and titles, for the report.
- **Line 8.** `datasets/` holds the final dataset CSV and its data dictionary.
- **Lines 9–10.** Creates the four folders. `parents=True` also creates any missing parent folder; `exist_ok=True` means no error if the folder already exists, so the cell can be run again safely.

```
12 SR = 16000
13 CLIP_SEC = 1.5    # spectrogram images are padded/cut to 1.5 s
14 N_MELS = 128
15 IMG_SIZE = 224
```

- **Line 12.** `SR = 16000`: every recording is resampled to 16,000 samples per second (16 kHz). The originals are 192 kHz, far more than speech needs. At 16 kHz the highest frequency that can be represented is half the sample rate, 8 kHz, which covers almost all the energy in speech, and every calculation runs about 12 times faster.
- **Line 13.** `CLIP_SEC = 1.5`: every spectrogram image covers exactly 1.5 seconds, so all CNN images share the same time scale. Shorter clips are padded with silence and longer ones are cut.
- **Line 14.** `N_MELS = 128`: the spectrograms have 128 frequency bands on the mel scale (explained in code cell 10).
- **Line 15.** `IMG_SIZE = 224`: the images are resized to 224 × 224 pixels, the size ResNet18 was trained on.

```
17 N_PCA = 5         # small because we only have 30 people
18 N_PERM = 1000
```

- **Line 17.** `N_PCA = 5`: before each classifier, the features are reduced to 5 combined features. Each model trains on only 29 people; with 67 (or 512) features and 29 examples, a model can easily fit noise, and 5 components keep it simple.
- **Line 18.** `N_PERM = 1000`: the permutation test (code cell 22) shuffles the labels 1000 times.

```
20 MIN_VALID = 40
21 GROUP_YEAR = "1st Year"
22 WORDS = ["Salamat", "Ambot", "Grabe", "Lisod", "Sayon", "Lingaw", "Kapoy", "Libog"]  # we
       ↪ added Libog (confused)
```

- **Line 20.** `MIN_VALID = 40`: the project sheet's minimum number of valid observations. Code cell 5 compares the real count against it.
- **Line 21.** The year level this group was assigned.
- **Line 22.** The eight response words allowed: the seven from the project sheet plus "Libog" (Bisaya for "confused"), which the group added. Code cell 5 flags any word not in this list.

```
24 # fill these in before submitting
25 session_cutoff = "TO FILL IN: the cutoff agreed before collection (e.g. Morning = classes
       ↪ ending before 12:00 PM)"
26 missing_reason = "TO FILL IN: reason participant numbers 003, 005, 007 are missing"
```

- **Line 24.** A reminder comment.
- **Lines 25–26.** Two placeholders that only the group can fill in: the Morning / Afternoon–Evening cutoff agreed before collection, and why participant numbers 003, 005 and 007 have no recording. Code cell 5 prints both and warns while either still starts with "TO FILL IN".

```
29 def save_fig(fig, name):
30     fig.savefig(out_dir / f"{name}.png", dpi=300, bbox_inches="tight")
```

- **Line 29.** Defines a small helper function, `save_fig`, so every figure is saved the same way.
- **Line 30.** Saves the figure as `RESULTSV3/<name>.png` at 300 dots per inch (print quality). `bbox_inches="tight"` trims the empty white margin around the figure. Every plotting cell calls `save_fig` before `plt.show()`, because some Jupyter setups clear the figure once it has been shown.

4 Loading the metadata and making the labels

Code cell 4

Notebook heading: Metadata and labels

This cell reads the metadata CSV, adds a few helper columns, and turns each participant's 1–5 ratings into the two-class labels the models predict. It is the first cell shown in demo part A.

Uses: `meta_csv`, `audio_dir`.

Creates: `meta` (the CSV as read), `df` (the working table, one row per recording), `meta_by_file` (the CSV looked up by filename).

```
1  meta = pd.read_csv(meta_csv)
2  df = meta.rename(columns={"Audio Filename": "file", "Valence": "valence", "Arousal":
       ↪ "arousal"}).copy()
3  df["participant"] = df["file"].str.split("_").str[0]
4  df["word"] = df["file"].str.replace(".wav", "", regex=False).str.split("_").str[-1]
5  df["path"] = [str(audio_dir / f) for f in df["file"]]
```

- **Line 1.** Reads the CSV into a DataFrame called `meta`. It has one row per participant and the columns Participant Code, Year Level, Class Activity, Session, Spoken Word, Self-Reported Feeling, Valence, Arousal, Audio Filename and Video Filename.
- **Line 2.** Makes the working copy `df` and renames three columns to short names: `file`, `valence`, `arousal`. `.copy()` makes `df` independent, so later changes to `df` never change `meta`.
- **Line 3.** Gets the participant code from the filename: `.str.split("_")` splits `Y1-001_Y1_AM_Kapoy.wav` into `["Y1-001", "Y1", "AM", "Kapoy.wav"]` and `.str[0]` keeps the first piece, `Y1-001`. The `.str` part applies the string operation to every row at once.
- **Line 4.** Gets the spoken word: first removes `.wav` (with `regex=False`, so the dot is a plain dot and not "any character"), then splits on _ and keeps the last piece, `Kapoy`.
- **Line 5.** Builds the full path of each recording, for example `audio/Y1-001_Y1_AM_Kapoy.wav`, as text.

```
7  # 4-5 -> positive / high, 1-3 -> non-positive / low-mod
8  df["valence_class"] = np.where(df["valence"] > 3, "positive", "non-positive")
9  df["arousal_class"] = np.where(df["arousal"] > 3, "high", "low-mod")
```

- **Line 7.** A comment stating the rule used on the next two lines.
- **Line 8.** `np.where(condition, a, b)` gives `a` where the condition is true and `b` where it is false. Valence above 3 (that is, 4 or 5) becomes "positive"; 1, 2 or 3 becomes "non-positive".
- **Line 9.** The same rule for arousal: 4 or 5 is "high", 1 to 3 is "low-mod" (low to moderate).

Note. A rating of 3 (neutral) is put in the non-positive / low-mod class. That is why the non-positive class is mostly neutral ratings (13 of 18) and should not be described as "negative".

```
11 missing = [p for p in df["path"] if not Path(p).exists()]
12 print("missing audio:", missing)
13 df = df[~df["path"].isin(missing)].reset_index(drop=True)
14 meta_by_file = meta.set_index("Audio Filename")
```

- **Line 11.** Makes a list of every path that does not exist on disk.
- **Line 12.** Prints that list. An empty list `[]` means every recording was found.
- **Line 13.** Keeps only the rows whose audio exists. ~ means "not", so `~df["path"].isin(missing)`

is true for rows not in the missing list. `reset_index(drop=True)` renumbers the rows 0, 1, 2, ... again; later cells look recordings up by these numbers.

- **Line 14.** `meta_by_file` is the same CSV but indexed by audio filename, so any column can be looked up for a given file with `.map()` (used in code cells 7, 8 and 17).

```
16 print(len(df), "recordings from", df["participant"].nunique(), "participants")
17 print(df["valence_class"].value_counts().to_dict())
18 print(df["arousal_class"].value_counts().to_dict())
19 print(df["word"].value_counts().to_dict())
```

- **Line 16.** Prints how many recordings and how many different participants there are.
- **Lines 17–19.** Prints the class counts for valence and arousal and the count of each spoken word. `value_counts()` counts each distinct value and `.to_dict()` prints the result on one line.

Expected output with your data. `missing audio: []`, then `30 recordings from 30 participants`, `{'non-positive': 18, 'positive': 12}`, `{'low-mod': 20, 'high': 10}`, and Kapoy 13, Grabe, Lingaw, Lisod, Salamat and Sayon 3 each, Libog 2.

5 Checking the metadata against the protocol

Code cell 5

Notebook heading: Checking the metadata

This cell checks every row of the metadata against the rules of the project sheet: the filename pattern, the word list, the year and session, the allowed activities, the 1–5 ratings, duplicate participants, and the 40-observation minimum. Every problem found is collected in a list and shown at the end. It is part of demo A.

Uses: `df`, `WORDS`, `GROUP_YEAR`, `MIN_VALID`, `session_cutoff`, `missing_reason`.
Creates: `problems` (list of issues), `gaps` (missing participant numbers).
Saves to the results folder: `metadata_issues.csv`, only if at least one problem is found.

```
1 activities = ["Lecture/Discussion", "Laboratory Activity", "Programming/Coding Activity",
      ↪ "Quiz", "Examination",
2               "Presentation/Reporting", "Group Activity", "Recitation",
      ↪ "Project/Practical Activity", "Other"]
3 years = {"Y1": "1st Year", "Y2": "2nd Year", "Y3": "3rd Year", "Y4": "4th Year"}
4 sessions = {"AM": "Morning", "PM": "Afternoon-Evening"}
5 name_pattern = re.compile(r"^(Y[1-4])-(\d{3})_(Y[1-4])_(AM|PM)_([A-Za-z]+)\.wav$")
```

- **Lines 1–2.** The ten Class Activity categories allowed by the project sheet.
- **Line 3.** A dictionary that turns the code in the filename (`Y1`) into the text used in the metadata (`1st Year`).
- **Line 4.** The same for the session: `AM` is Morning and `PM` is Afternoon-Evening.
- **Line 5.** The filename pattern as a regular expression. Read from left to right: `^` start of the name; `(Y[1-4])` the year in the participant code; `-`; `(\d{3})` exactly three digits (the participant number); _; `(Y[1-4])` the year level again; _; `(AM|PM)` the session; _; `([A-Za-z]+)` the word, letters

only; `\.wav$` the literal ending ".wav". Each pair of brackets is a "group" that can be read out afterwards. The `r` before the string keeps the backslashes as they are.

```
7  problems = []
8  for _, r in df.iterrows():
9      m = name_pattern.match(r["file"])
10     if m is None:
11         problems.append((r["file"], "filename doesn't follow Code_Year_Session_Word.wav"))
12         continue
13     code_year, _, year, ses, word = m.groups()
```

- **Line 7.** An empty list that will collect `(file, problem)` pairs.
- **Line 8.** Goes through the table one row at a time. `iterrows()` gives the row number (ignored, hence _) and the row `r`.
- **Line 9.** Tries to match the filename against the pattern. The result is a match object, or `None` if the name does not fit.
- **Lines 10–12.** If it does not fit, records the problem and uses `continue` to skip to the next row, because the remaining checks need the pieces of the name.
- **Line 13.** Reads the five groups out of the match: year in the code, participant number (not needed, so _), year level, session, and word.

```
14     if word not in WORDS:
15         problems.append((r["file"], f"{word} is not in our word list"))
16     if code_year != year:
17         problems.append((r["file"], "year in code and filename don't match"))
18     if years[year] != GROUP_YEAR:
19         problems.append((r["file"], f"not {GROUP_YEAR}"))
20     if r["Year Level"] != years[year]:
21         problems.append((r["file"], "Year Level column doesn't match filename"))
22     if r["Session"] != sessions[ses]:
23         problems.append((r["file"], "Session column doesn't match AM/PM"))
24     if r["Spoken Word"] != word:
25         problems.append((r["file"], "Spoken Word column doesn't match filename"))
26     if r["Class Activity"] not in activities:
27         problems.append((r["file"], f"unknown activity: {r['Class Activity']}"))
28     for col in ["valence", "arousal"]:
29         if r[col] not in [1, 2, 3, 4, 5]:
30             problems.append((r["file"], f"{col} should be 1-5, got {r[col]}"))
```

- **Lines 14–15.** The word must be one of the eight in `WORDS`.
- **Lines 16–17.** The year in the participant code (`Y1-001`) must equal the year in the filename (`_Y1_`).
- **Lines 18–19.** The year must be the group's assigned year, 1st Year.
- **Lines 20–21.** The Year Level column must agree with the filename.
- **Lines 22–23.** The Session column must agree with AM/PM in the filename.
- **Lines 24–25.** The Spoken Word column must agree with the word in the filename.

- **Lines 26–27.** The Class Activity must be one of the ten allowed categories.
- **Lines 28–30.** Valence and arousal must each be a whole number from 1 to 5. An empty cell, 2.5 or 6 would be flagged.

```
32 repeated = df["participant"][df["participant"].duplicated()].unique().tolist()
33 if repeated:
34     problems.append(("(several)", f"participants with more than one recording:
       ↪ {repeated}"))
```

- **Line 32.** Finds participant codes that appear more than once. `duplicated()` marks the second and later copies; `unique().tolist()` lists each repeated code once.
- **Lines 33–34.** If any code repeats, adds one problem that names them. The protocol here is one recording per participant.

```
36 numbers = sorted(int(p.split("-")[1]) for p in df["participant"])
37 gaps = [n for n in range(1, max(numbers) + 1) if n not in numbers]
```

- **Line 36.** Takes the number part of each code ("Y1-014" → "014" → 14) and sorts them.
- **Line 37.** Lists every number from 1 up to the largest one that has no recording. For this data that is 3, 5 and 7.

```
39 print(f"valid observations: {len(df)} (need {MIN_VALID})")
40 if len(df) < MIN_VALID:
41     print(f"-> {MIN_VALID - len(df)} short, put this in the limitations")
42 if gaps:
43     print("missing participant numbers:", [f"{n:03d}" for n in gaps])
44     print("reason:", missing_reason)
```

- **Line 39.** Prints the number of valid observations next to the required 40.
- **Lines 40–41.** If below 40, prints how many are missing as a reminder for the limitations.
- **Lines 42–44.** If there are gaps, prints them as three-digit codes (`{n:03d}` pads with zeros, so 3 becomes 003) and prints the reason written in the setup cell.

```
46 if problems:
47     problems_df = pd.DataFrame(problems, columns=["file", "issue"])
48     display(problems_df)
49     problems_df.to_csv(out_dir / "metadata_issues.csv", index=False, encoding="utf-8")
50 else:
51     print("no problems with the metadata or filenames")
```

- **Line 46.** If at least one problem was found…
- **Lines 47–49.** …turns the list into a two-column table, shows it, and saves it as `metadata_issues.csv`.
- **Lines 50–51.** Otherwise prints that there are no problems.

Note. `metadata_issues.csv` is only written when there are problems. If an old copy from an earlier run is still in the folder, it is not deleted.

```
53 print("session cutoff:", session_cutoff)
54 if session_cutoff.startswith("TO FILL IN") or missing_reason.startswith("TO FILL IN"):
55     print("-> still need to fill in session_cutoff / missing_reason in the setup cell")
```

- **Line 53.** Prints the session cutoff from the setup cell.
- **Lines 54–55.** If either placeholder still starts with "TO FILL IN", prints a reminder to fill it in.

Expected output with your data. `valid observations: 30 (need 40), -> 10 short, put this in the limitations, missing participant numbers: ['003', '005', '007'], no problems with the metadata or filenames`, and the reminder about the two placeholders until they are filled in.

6 Matching files to the metadata

Code cell 6

Notebook heading: Do the files match the metadata?

This cell answers demo part A directly: how many participants there are, whether every metadata row has its audio and video file, whether any file on disk is missing from the metadata, and whether every label is valid. The videos are only counted here; they are not analysed anywhere in the notebook.

Uses: `meta`, `audio_dir`, `video_dir`.
Creates: `listed`, `wavs`, `mp4s`, `files` (one row per participant with the checks), `counts`.
Saves to the results folder: `file_correspondence_check.csv`.

```
1 listed = meta["Audio Filename"].astype(str).tolist()
2 wavs = sorted(p.name for p in audio_dir.glob("*.wav"))
3 mp4s = sorted(p.name for p in video_dir.glob("*.mp4")) if video_dir.exists() else []
```

- **Line 1.** The audio filenames listed in the metadata, as a Python list of text.
- **Line 2.** The names of all `.wav` files actually in `audio/`. `glob("*.wav")` finds them; `p.name` keeps only the filename without the folder.
- **Line 3.** The same for `.mp4` files in `video/`. If the folder does not exist, it uses an empty list instead of failing.

```
5  files = pd.DataFrame({
6      "participant": meta["Participant Code"],
7      "audio_file": listed,
8      "wav_found": [f in wavs for f in listed],
9      "mp4_found": [f.replace(".wav", ".mp4") in mp4s for f in listed],
10     "valence": meta["Valence"].values,
11     "arousal": meta["Arousal"].values,
12 })
13 files["labels_valid"] = files["valence"].isin([1, 2, 3, 4, 5]) &
```

```
    ↪ files["arousal"].isin([1, 2, 3, 4, 5])
files["valence_class"] = np.where(files["valence"] > 3, "positive", "non-positive")
files["arousal_class"] = np.where(files["arousal"] > 3, "high", "low-mod")
```

- **Lines 5–12.** Builds a table with one row per metadata row: the participant code, the audio filename, whether that WAV exists (`wav_found`), whether the matching MP4 exists (`mp4_found`, made by swapping `.wav` for `.mp4`), and the two ratings. `.values` takes the plain numbers out of the column.
- **Line 13.** `labels_valid` is true only when both ratings are whole numbers from 1 to 5. `isin` checks membership for every row; `&` means "and" for each row.
- **Lines 14–15.** The same class rule as code cell 4, repeated so this table is complete on its own.

```
counts = pd.Series({
    "metadata rows": len(files),
    "participants": files["participant"].nunique(),
    "wav files": len(wavs),
    "rows with a wav": int(files["wav_found"].sum()),
    "mp4 files": len(mp4s),
    "rows with an mp4": int(files["mp4_found"].sum()),
    "rows with valid labels": int(files["labels_valid"].sum()),
})
display(counts.to_frame("count"))
```

- **Lines 17–25.** Seven counts in one small table: metadata rows, different participants, WAV files on disk, rows with a WAV, MP4 files on disk, rows with an MP4, rows with valid labels. Summing a true/false column counts the trues.
- **Line 26.** Shows the counts as a one-column table.

```
print("wav files not in the metadata:", sorted(set(wavs) - set(listed)) or "none")
print("mp4 files not in the metadata:", sorted(set(mp4s) - {f.replace(".wav", ".mp4") for
    ↪ f in listed}) or "none")
```

- **Line 28.** Files on disk that are not in the metadata: the set of WAV names minus the set of listed names. An empty list counts as false in Python, so `or "none"` prints the word "none" instead of `[]`.
- **Line 29.** The same for MP4 files, compared against the MP4 names the metadata implies.

```
print("\nlabels:")
print(files["valence_class"].value_counts().to_dict())
print(files["arousal_class"].value_counts().to_dict())
display(files)
files.to_csv(out_dir / "file_correspondence_check.csv", index=False)
```

- **Lines 31–33.** Prints the label counts.
- **Line 34.** Shows the full table, one row per participant.
- **Line 35.** Saves it as `file_correspondence_check.csv`.

Expected output with your data. all seven counts are 30; no unmatched WAV or MP4 files; valence `{'non-positive': 18, 'positive': 12}` and arousal `{'low-mod': 20, 'high': 10}`.

7 Exploratory analysis of the ratings (RQ2)

Code cell 7

Notebook heading: Exploratory analysis

Before any modelling, this cell describes the ratings: overall, and split by year level, class activity, session and spoken word. Everything here is descriptive; no statistical tests are run.

Uses: `df`, `meta_by_file`.

Creates: `summary`, `tbl` (per group), `word_by_activity`.

Saves to the results folder: `exploratory_rating_summary.csv`, `exploratory_rating_distribution.png`, `exploratory_<column>_summary.csv` and `exploratory_<column>.png` for each grouping, `exploratory_word_by_activity_counts.csv`.

```
1  for col in ["Year Level", "Class Activity", "Session"]:
2      df[col] = df["file"].map(meta_by_file[col])
```

- **Lines 1–2.** Fills the Year Level, Class Activity and Session columns of `df` by looking up each recording's filename in `meta_by_file`. These columns were already copied into `df` in code cell 4; looking them up by filename makes sure each value lines up with the right recording even if some rows had been removed.

```
4  summary = df[["valence", "arousal"]].describe().round(2)
5  display(summary)
6  summary.to_csv(out_dir / "exploratory_rating_summary.csv")
```

- **Line 4.** `describe()` gives count, mean, standard deviation, minimum, quartiles and maximum of the two ratings, rounded to 2 decimals.
- **Line 5.** Shows the table.
- **Line 6.** Saves it.

```
8  fig, ax = plt.subplots(1, 2, figsize=(10, 3.5))
9  sns.countplot(x="valence", data=df, order=[1, 2, 3, 4, 5], ax=ax[0])
10 sns.countplot(x="arousal", data=df, order=[1, 2, 3, 4, 5], ax=ax[1])
11 ax[0].set_title("Valence ratings")
12 ax[1].set_title("Arousal ratings")
13 plt.tight_layout()
14 save_fig(fig, "exploratory_rating_distribution")
15 plt.show()
```

- **Line 8.** Creates a figure with two side-by-side panels (`ax[0]` and `ax[1]`), 10 × 3.5 inches.
- **Lines 9–10.** Bar charts of how many participants gave each rating. `order=[1, 2, 3, 4, 5]` keeps the bars in order and includes every rating.
- **Lines 11–12.** Panel titles.

- **Line 13.** `tight_layout()` adjusts spacing so labels do not overlap.
- **Line 14.** Saves the figure with the helper from code cell 3.
- **Line 15.** Shows it in the notebook.

```
17 for col in ["Year Level", "Class Activity", "Session", "word"]:
18     name = col.lower().replace(" ", "_")
19     tbl = df.groupby(col)[["valence", "arousal"]].agg(["count", "mean", "std"]).round(2)
20     print("\n" + col)
21     display(tbl)
22     tbl.to_csv(out_dir / f"exploratory_{name}_summary.csv")
23     if df[col].nunique() < 2:
24         print("only one value, nothing to compare")
25         continue
```

- **Line 17.** Repeats the following for four ways of grouping the participants.
- **Line 18.** A file-friendly version of the column name ("Class Activity" → `class_activity`).
- **Line 19.** `groupby(col)` splits the participants by the values of that column; `.agg(["count", "mean", "std"])` gives, for each group, how many people, their average rating and the spread, for both valence and arousal.
- **Line 20.** Prints the column name as a heading. `"\n"` adds an empty line before it.
- **Lines 21–22.** Shows and saves the table.
- **Lines 23–25.** If the column has only one value (Year Level: everyone is 1st Year), there is nothing to compare, so it prints a note and `continue` skips the plot.

Note. The standard deviation of a group with one person (Laboratory Activity) is empty (NaN), because a spread cannot be computed from one value.

```
26     fig, ax = plt.subplots(1, 2, figsize=(11, 3.5))
27     for a, v in zip(ax, ["valence", "arousal"]):
28         sns.stripplot(data=df, x=col, y=v, ax=a, jitter=True, size=8)
29         a.set_ylim(0.5, 5.5)
30         a.set_title(f"{v.capitalize()} by {col}")
31         a.tick_params(axis="x", rotation=30)
32     plt.tight_layout()
33     save_fig(fig, f"exploratory_{name}")
34     plt.show()
```

- **Line 26.** Two panels again.
- **Line 27.** Loops over the two panels together with the two ratings.
- **Line 28.** A strip plot: one dot per participant, the group on the x-axis and the rating on the y-axis. `jitter=True` spreads dots sideways so identical ratings do not hide each other.
- **Line 29.** Fixes the y-axis to 0.5–5.5 so every plot uses the same scale.
- **Line 30.** Title such as "Valence by Session".

- **Line 31.** Tilts the x labels 30° so long activity names fit.
- **Lines 32–34.** Spacing, save as `exploratory_<column>.png`, show.

```
36 word_by_activity = pd.crosstab(df["Class Activity"], df["word"])
37 print("\nword x activity")
38 display(word_by_activity)
39 word_by_activity.to_csv(out_dir / "exploratory_word_by_activity_counts.csv")
```

- **Line 36.** `crosstab` counts how many recordings fall in each combination of class activity (rows) and word (columns). It shows whether some words were mostly said after one kind of class.
- **Lines 37–39.** Prints a heading, shows and saves the table.

Expected output with your data. Quiz 11, Examination 9, Lecture/Discussion 4, Project/Practical 3, Presentation/Reporting 2, Laboratory 1. Morning has 13 people (mean arousal 3.31) and Afternoon–Evening 17 (mean arousal 2.41).

8 RQ1: how students felt after class

Code cell 8

Notebook heading: RQ1: how did students feel after class?

This cell answers RQ1: what affective states the students reported. It counts the free-text feelings, cross-tabulates the two classes, and draws the ratings on the valence–arousal plane twice: as a scatter plot and as an exact-count heatmap.

Uses: `df`, `meta_by_file`, `SEED`.

Creates: `feelings`, `class_counts`, `grid`.

Saves to the results folder: `rq1_self_reported_feelings_counts.csv`, `rq1_valence_class_by_arousal_class_counts.csv`, `rq1_valence_arousal_scatter.png`, `rq1_valence_by_arousal_rating_counts.csv`, `rq1_valence_arousal_counts.png`.

```
1 df["feeling"] = df["file"].map(meta_by_file["Self-Reported
      ↪ Feeling"]).astype(str).str.strip().str.lower()
2 feelings = df["feeling"].value_counts().to_frame("count")
3 display(feelings)
4 feelings.to_csv(out_dir / "rq1_self_reported_feelings_counts.csv", encoding="utf-8")
```

- **Line 1.** Looks up each participant's free-text feeling, converts it to text, removes spaces at the ends and makes it lower case, so "Kapoy" and "kapoy " count as the same answer.
- **Line 2.** Counts each distinct answer and turns the counts into a one-column table.
- **Lines 3–4.** Shows and saves it.

```
6 class_counts = pd.crosstab(df["valence_class"], df["arousal_class"])
7 display(class_counts)
8 class_counts.to_csv(out_dir / "rq1_valence_class_by_arousal_class_counts.csv",
      ↪ encoding="utf-8")
```

- **Line 6.** A 2 × 2 table: valence class (rows) against arousal class (columns).
- **Lines 7–8.** Shows and saves it.

```
10 # small random jitter so points with the same rating don't sit on top of each other
11 jitter = np.random.default_rng(SEED)
12 fig = plt.figure(figsize=(5.5, 5))
13 sns.scatterplot(x=df["valence"] + jitter.uniform(-0.12, 0.12, len(df)),
14                 y=df["arousal"] + jitter.uniform(-0.12, 0.12, len(df)),
15                 hue=df["word"], s=110, alpha=0.8)
```

- **Line 10.** Explains why the next lines add jitter.
- **Line 11.** Creates a random generator seeded with 42, so the jitter is identical in every run.
- **Line 12.** A new figure, 5.5 × 5 inches.
- **Lines 13–15.** A scatter plot with valence on x and arousal on y. Each rating gets a small random shift between −0.12 and +0.12 (`uniform`), because many participants gave the same pair of ratings and would otherwise sit exactly on top of each other. The shift is small enough that every point stays clearly next to its true rating. `hue=df["word"]` colours the points by spoken word; `s` is the marker size and `alpha` the transparency.

```
16 plt.xlim(0.5, 5.5)
17 plt.ylim(0.5, 5.5)
18 plt.xlabel("Valence (1 = very unpleasant, 5 = very pleasant)")
19 plt.ylabel("Arousal (1 = very low, 5 = very high)")
20 plt.title("Self-reported valence and arousal (jittered), coloured by word")
21 plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left", fontsize=8)
22 plt.tight_layout()
23 save_fig(fig, "rq1_valence_arousal_scatter")
24 plt.show()
```

- **Lines 16–17.** Fixes both axes to 0.5–5.5.
- **Lines 18–20.** Axis labels and the title.
- **Line 21.** Puts the legend outside the plot on the right (`bbox_to_anchor=(1.02, 1)`), so it does not cover points.
- **Lines 22–24.** Spacing, save, show.

```
26 grid = pd.crosstab(df["arousal"], df["valence"]).reindex(index=[1, 2, 3, 4, 5],
       ↪ columns=[1, 2, 3, 4, 5], fill_value=0)
27 grid.to_csv(out_dir / "rq1_valence_by_arousal_rating_counts.csv", encoding="utf-8")
```

- **Line 26.** Counts the participants in every combination of arousal (rows) and valence (columns). `reindex` makes sure all 25 combinations of 1–5 appear, filling those nobody chose with 0.
- **Line 27.** Saves the counts.

```
29 fig = plt.figure(figsize=(5.2, 4.4))
30 sns.heatmap(grid.sort_index(ascending=False), annot=True, fmt="d", cmap="Blues",
       ↪ cbar_kws={"label": "Number of recordings"})
31 plt.xlabel("Valence (1 = very unpleasant, 5 = very pleasant)")
32 plt.ylabel("Arousal (1 = very low, 5 = very high)")
33 plt.title("Recordings per valence-arousal rating")
34 plt.tight_layout()
35 save_fig(fig, "rq1_valence_arousal_counts")
36 plt.show()
```

- **Line 29.** A new figure.
- **Line 30.** Draws the counts as a heatmap. `sort_index(ascending=False)` puts arousal 5 at the top, so "more energy" is up. `annot=True` writes each count in its square and `fmt="d"` formats it as a whole number. The colour bar is labelled.
- **Lines 31–33.** Labels and title.
- **Lines 34–36.** Spacing, save, show.

Expected output with your data. 25 of 30 rated valence 3 or higher; every unpleasant rating (1 or 2) came with arousal 3 or lower; the most common combinations were (valence 3, arousal 3) and (4, 4) with 5 people each. Valence × arousal classes: non-positive/low-mod 15, non-positive/high 3, positive/low-mod 5, positive/high 7.

9 Audio quality report

Code cell 9

Notebook heading: Audio quality

Before any processing, this cell checks every original WAV file: sample rate, number of channels, length, loudness, clipping and how much of it is silence. Problem files get a flag so they can be reviewed by ear.

Uses: `df`.
Creates: `quality` (one row per file).
Saves to the results folder: `audio_quality_report.csv`.

```
1 quality = []
2 for _, r in df.iterrows():
3     y, sr = librosa.load(r["path"], sr=None, mono=False)
4     channels = 1 if y.ndim == 1 else y.shape[0]
5     if y.ndim > 1:
6         y = y.mean(axis=0)
```

- **Line 1.** An empty list for the results.
- **Line 2.** Goes through every recording.
- **Line 3.** Loads the file exactly as recorded: `sr=None` keeps the original sample rate (no resampling) and `mono=False` keeps both channels. `y` holds the samples as numbers between −1 and 1; `sr` is the sample rate.

- **Line 4.** A mono file is a one-dimensional array; a stereo file has one row per channel, so the number of rows is the number of channels.
- **Lines 5–6.** If there is more than one channel, averages them into one, so the checks below work the same for every file.

```
7       dur = len(y) / sr
8       peak = float(np.max(np.abs(y)))
9       clipped = float(np.mean(np.abs(y) >= 0.99))
10      rms_db = float(20 * np.log10(np.sqrt(np.mean(y ** 2)) + 1e-9))
11      active = float(sum(e - s for s, e in librosa.effects.split(y, top_db=30)) / len(y))
        ↪ if len(y) else 0.0
```

- **Line 7.** Duration in seconds: number of samples divided by samples per second.
- **Line 8.** Peak: the largest absolute sample value. 1.0 is the loudest a WAV file can store.
- **Line 9.** Fraction of samples at or above 0.99 in absolute value, that is, at the ceiling. A noticeable fraction means clipping (distortion from recording too loud).
- **Line 10.** Overall loudness in decibels relative to full scale (dBFS), from the root-mean-square of all samples. The tiny `1e-9` prevents taking the logarithm of zero for a completely silent file.
- **Line 11.** `librosa.effects.split(y, top_db=30)` finds the stretches that are not silent, treating anything more than 30 dB below the loudest part of the file as silence. The sum of their lengths divided by the file length is the fraction of the file that contains sound.

Formula. loudness in dB from the samples $x_1, \ldots, x_N$:

$$\text{rms_dB} = 20 \log_{10}\left(\sqrt{\tfrac{1}{N}\textstyle\sum_n x_n^2} + 10^{-9}\right)$$

```
13      flags = []
14      if dur < 0.3:
15          flags.append("very short")
16      if dur > 5:
17          flags.append("long (may contain extra speech)")
18      if peak < 0.05:
19          flags.append("very quiet")
20      if clipped > 0.001:
21          flags.append("clipping")
22      if active < 0.1:
23          flags.append("mostly silence")
```

- **Line 13.** An empty list of flags for this file.
- **Lines 14–15.** Shorter than 0.3 s: probably cut off.
- **Lines 16–17.** Longer than 5 s: probably contains more than one word.
- **Lines 18–19.** Peak below 0.05: recorded very quietly.
- **Lines 20–21.** More than 0.1% of samples at the ceiling: clipping.
- **Lines 22–23.** Less than 10% of the file is sound: mostly silence.

```
25      quality.append({"file": r["file"], "sample_rate": sr, "channels": channels,
        ↪ "duration_s": round(dur, 2),
26                      "peak": round(peak, 3), "rms_dB": round(rms_db, 1), "clipping_frac":
        ↪ round(clipped, 4),
27                      "active_ratio": round(active, 2), "flags": "; ".join(flags)})
```

- **Lines 25–27.** Adds one result row for this file, with values rounded for readability and the flags joined into one text such as "very quiet; clipping" (empty if none).

```
29  quality = pd.DataFrame(quality)
30  quality.to_csv(out_dir / "audio_quality_report.csv", index=False)
31  print("sample rates:", sorted(quality["sample_rate"].unique()), "| channels:",
        ↪ sorted(quality["channels"].unique()))
32  display(quality.describe().round(2))
33  print("flagged:", (quality["flags"] != "").sum())
34  display(quality[quality["flags"] != ""])
```

- **Line 29.** Turns the list of rows into a table.
- **Line 30.** Saves it.
- **Line 31.** Prints the distinct sample rates and channel counts found, to confirm every file was recorded the same way.
- **Line 32.** Summary statistics of all numeric columns.
- **Lines 33–34.** How many files have at least one flag, and the flagged rows.

Expected output with your data. `sample rates: [192000] | channels: [2]`, durations from 0.68 s to 2.01 s, and `flagged: 0`.

10 Preprocessing and feature extraction

Code cell 10

Notebook heading: Feature extraction

This is the most important cell. It defines how a recording is cleaned and turned into numbers, then applies it to all 30 recordings. It produces two kinds of input for the models:

- **67 handcrafted features** per recording: timing, pitch, energy, zero-crossing rate, spectral centroid and MFCCs.
- **A plain mel-spectrogram image** per recording, which code cell 16 turns into 512 numbers with a pretrained network.

Many librosa functions work on short overlapping **frames**. Unless a different value is given, a frame is 2048 samples (128 ms at 16 kHz) and the next frame starts 512 samples (32 ms) later; this step is called the **hop**. A frame-level feature therefore gives one value every 32 ms, and the notebook summarises those values with their mean and standard deviation to get a fixed number of features per recording.

Uses: `df`, `SR`, `CLIP_SEC`, `N_MELS`, `IMG_SIZE`, `spec_dir`.

Creates: the functions `load_audio`, `mean_std`, `get_features`, `pad_or_cut`, `mel_db`, `save_spectrogram_image`, `plot_spectrogram`; and `clips` (processed audio per recording), `img_paths`, `feat_df` (30 × 67 table), `X_hand` (the same as an array).

Saves to the results folder: 30 PNG images in `SPECTROGRAMS/` and `acoustic_features.csv`.

```
1  def load_audio(path):
2      y, _ = librosa.load(path, sr=SR, mono=True)
3      y, _ = librosa.effects.trim(y, top_db=25)
4      if len(y) < 1024:
5          y = np.pad(y, (0, 1024 - len(y)))
6      return librosa.util.normalize(y)
```

`load_audio` is the preprocessing applied to every recording before any feature is computed.

- **Line 1.** Defines the function; `path` is the WAV file to load.
- **Line 2.** Loads the file, averages the two channels into one (`mono=True`) and resamples from 192 kHz to 16 kHz (`sr=SR`) with librosa's high-quality resampler. The second value returned (the sample rate) is not needed, hence _.
- **Line 3.** Removes silence at the start and end: anything quieter than 25 dB below the loudest part of the clip is cut. Silence in the middle is kept.
- **Lines 4–5.** If the trimmed clip is shorter than 1024 samples (64 ms), pads it with zeros up to 1024, so every feature function has at least one full frame to work on. None of the 30 recordings is this short; it is a safety check.
- **Line 6.** Peak-normalises: divides every sample by the largest absolute value, so the loudest point becomes exactly 1.0.

Note. Because of peak normalisation, the features describe how loudness changes within the word, not how loudly the person spoke. Absolute loudness, often a strong cue for arousal, is lost on purpose: microphone distance was not controlled, so absolute loudness would mostly measure distance to the phone.

```
9  def mean_std(x, name):
10     return {f"{name}_mean": float(np.mean(x)), f"{name}_std": float(np.std(x))}
```

- **Line 9.** A helper that summarises a frame-level feature.
- **Line 10.** Returns two named values: the mean over all frames and the standard deviation over all frames (for example `rms_mean` and `rms_std`). `np.std` here is the population standard deviation (dividing by N).

```
13 def get_features(y):
14     f = {"duration": len(y) / SR}
```

- **Line 13.** `get_features` takes one preprocessed clip `y` and returns a dictionary of 67 named numbers.
- **Line 14.** Starts the dictionary with the duration in seconds (samples ÷ 16,000) after trimming.

```
16      segs = librosa.effects.split(y, top_db=30)
17      f["n_segments"] = len(segs)
18      f["active_ratio"] = float(sum(e - s for s, e in segs) / len(y))
19      rms = librosa.feature.rms(y=y)[0]
20      f["rms_peak_pos"] = float(np.argmax(rms) / max(len(rms) - 1, 1))
```

Timing and position of the loudest point.

- **Line 16.** Finds the non-silent stretches inside the clip (more than 30 dB below the loudest part counts as silence). The result is a list of `[start, end]` sample positions.
- **Line 17.** `n_segments`: how many separate bursts of sound there are. A word with a clear pause between syllables gives 2 or more.
- **Line 18.** `active_ratio`: the fraction of the clip that is sound rather than pause.
- **Line 19.** Computes the RMS (root-mean-square) energy of each frame, a measure of how loud each 128 ms piece is. `[0]` takes the single row out of the two-dimensional result.
- **Line 20.** `rms_peak_pos`: where the loudest frame is, from 0 (start) to 1 (end). `argmax` gives the frame number of the maximum; dividing by the number of frames minus one scales it to 0–1. `max(…, 1)` prevents a division by zero if there is only one frame.

Formula. RMS energy of frame t with samples $x_t[1..N]$: $\mathrm{RMS}_t = \sqrt{\frac{1}{N}\sum_{n=1}^{N} x_t[n]^2}$.

```
22      f0, voiced, _ = librosa.pyin(y, fmin=65, fmax=500, sr=SR)
23      f0 = f0[~np.isnan(f0)]
24      if len(f0):
25          f.update({"f0_mean": f0.mean(), "f0_std": f0.std(), "f0_range": f0.max() -
        ↪ f0.min()})
26      else:
27          f.update({"f0_mean": 0.0, "f0_std": 0.0, "f0_range": 0.0})
28      f["voiced_ratio"] = float(np.mean(voiced))
```

Pitch (fundamental frequency, f0), the main carrier of intonation.

- **Line 22.** `librosa.pyin` is the probabilistic YIN pitch tracker. It estimates the pitch of each frame between 65 Hz and 500 Hz, which covers normal speaking pitch for both men and women. It returns three arrays: the pitch per frame (`NaN`, "not a number", for frames with no voice), a true/false voiced flag per frame, and the probability of voicing (not used, hence _).
- **Line 23.** Keeps only the frames with a pitch value (`~np.isnan` means "is a number").
- **Lines 24–25.** If any frame was voiced: `f0_mean` (average pitch in Hz), `f0_std` (how much the pitch moves) and `f0_range` (highest minus lowest pitch).
- **Lines 26–27.** If no frame was voiced, all three are set to 0. This is a placeholder, not a real pitch.
- **Line 28.** `voiced_ratio`: the fraction of frames with voice. The mean of a true/false array is the fraction of trues.

```
30      f.update(mean_std(rms, "rms"))
31      f["rms_max"] = float(rms.max())
```

```
32      f.update(mean_std(librosa.feature.zero_crossing_rate(y)[0], "zcr"))
33      f.update(mean_std(librosa.feature.spectral_centroid(y=y, sr=SR)[0], "centroid"))
```

Energy and the shape of the spectrum.

- **Line 30.** Adds `rms_mean` and `rms_std` from the frame energies computed on line 19.
- **Line 31.** `rms_max`: the energy of the loudest frame.
- **Line 32.** Zero-crossing rate: in each frame, the fraction of neighbouring samples where the signal changes sign. It is high for noisy, hissing sounds (s, h, breath) and low for voiced vowels. Adds `zcr_mean` and `zcr_std`.
- **Line 33.** Spectral centroid: the average frequency of the frame weighted by how strong each frequency is, in Hz. Higher values mean a brighter, sharper sound. Adds `centroid_mean` and `centroid_std`.

Formula. zero-crossing rate of frame t: $\mathrm{ZCR}_t = \frac{1}{N}\sum_n \mathbf{1}\big[\,\mathrm{sign}(x_t[n]) \neq \mathrm{sign}(x_t[n-1])\big]$. Spectral centroid, with $|X_t(k)|$ the strength of frequency f_k: $C_t = \dfrac{\sum_k f_k\,|X_t(k)|}{\sum_k |X_t(k)|}$.

```
35      mfcc = librosa.feature.mfcc(y=y, sr=SR, n_mfcc=13)
36      d1 = librosa.feature.delta(mfcc, width=5, mode="nearest")
37      d2 = librosa.feature.delta(mfcc, order=2, width=5, mode="nearest")
38      for i in range(13):
39          f.update(mean_std(mfcc[i], f"mfcc{i}"))
40          f[f"dmfcc{i}_std"] = float(np.std(d1[i]))
41          f[f"ddmfcc{i}_std"] = float(np.std(d2[i]))
42      return f
```

MFCCs (mel-frequency cepstral coefficients), the standard description of the sound's spectral shape.

- **Line 35.** Computes 13 MFCCs per frame. librosa first makes a 128-band mel spectrogram (frequency bands spaced the way the ear resolves pitch: about evenly up to 1 kHz and logarithmically above), converts it to decibels, then applies a discrete cosine transform and keeps the first 13 coefficients. Coefficient 0 mostly follows overall level; the higher coefficients describe the shape of the spectrum, which depends on the vowel, the consonants and the speaker's voice. The result has 13 rows and one column per frame.
- **Line 36.** Delta MFCCs: how fast each coefficient is changing over time, estimated over a window of 5 frames. `mode="nearest"` repeats the edge frames at the start and end so the window always has 5 values.
- **Line 37.** Delta-delta MFCCs (`order=2`): how fast that change is changing, the "acceleration".
- **Line 38.** Loops over the 13 coefficients, numbered 0 to 12.
- **Line 39.** Mean and standard deviation of coefficient i over time: `mfcc0_mean`, `mfcc0_std`, and so on.
- **Line 40.** Standard deviation of its delta. Only the spread is kept, because the average slope over a whole word is close to zero.
- **Line 41.** Standard deviation of its delta-delta.

- **Line 42.** Returns the dictionary. Count: 15 timing, pitch, energy, ZCR and centroid features plus $13 \times 4 = 52$ MFCC features gives **67**.

Formula. MFCC c of frame t from the log mel energies $E_{t,m}$ in $M = 128$ bands (DCT type II): $c_t(i) \propto \sum_{m=0}^{M-1} E_{t,m} \cos\left[\frac{\pi i}{M}\left(m + \frac{1}{2}\right)\right]$, $i = 0, \ldots, 12$.

```
45 def pad_or_cut(y):
46     n = int(CLIP_SEC * SR)
47     return y[:n] if len(y) >= n else np.pad(y, (0, n - len(y)))
```

- **Line 45.** `pad_or_cut` makes every clip exactly 1.5 s long for the spectrogram images. It is not used for the 67 features.
- **Line 46.** 1.5 s × 16,000 = 24,000 samples.
- **Line 47.** Longer clips keep their first 24,000 samples; shorter clips get zeros added at the end. The zeros are the black area on the right of short clips' spectrograms.

```
50 def mel_db(y):
51     S = librosa.feature.melspectrogram(y=pad_or_cut(y), sr=SR, n_fft=1024,
       ↪ hop_length=128, n_mels=N_MELS)
52     return librosa.power_to_db(S, ref=np.max)
```

- **Line 50.** `mel_db` makes the mel spectrogram used for the images.
- **Line 51.** Fixes the clip to 1.5 s, then computes the power in 128 mel bands for frames of 1024 samples (64 ms) every 128 samples (8 ms). For 24,000 samples that gives a grid of 128 bands × 188 time steps.
- **Line 52.** Converts power to decibels relative to the strongest cell, so the loudest point is 0 dB and everything else is negative. librosa cuts the range off at 80 dB below the maximum, so values run from −80 to 0.

Formula. $S_{\text{dB}} = \max\left(10 \log_{10}(S / \max S),\ -80\right)$

```
55 def save_spectrogram_image(y, path):
56     # no axes, this is what the CNN sees
57     S = mel_db(y)
58     S = (S - S.min()) / (S.max() - S.min() + 1e-9)
59     rgb = (matplotlib.colormaps["magma"](S)[..., :3] * 255).astype(np.uint8)
60     Image.fromarray(np.flipud(rgb)).resize((IMG_SIZE, IMG_SIZE),
       ↪ Image.BILINEAR).save(path)
61     return str(path)
```

- **Line 55.** `save_spectrogram_image` turns a clip into the image the CNN sees.
- **Line 56.** A comment: no axes or labels are drawn, because they would become part of the image.
- **Line 57.** The dB spectrogram.
- **Line 58.** Rescales the values of this image to 0–1 (lowest value 0, highest 1). Because each image

is scaled on its own, brightness cannot be compared between clips.

- **Line 59.** Colours the values with the "magma" colour map (black → purple → orange → yellow), keeps the red, green and blue channels (`[..., :3]` drops transparency), and converts to whole numbers 0–255 as image files expect.
- **Line 60.** `np.flipud` flips the array upside down: row 0 of the array is the lowest frequency, but row 0 of an image is the top, so without the flip low frequencies would appear at the top. The image is then resized from 188 × 128 to 224 × 224 pixels (smooth bilinear resizing) and saved.
- **Line 61.** Returns the file path as text, so the CNN cell knows where the image is.

```
64 def plot_spectrogram(ax, y, title):
65     img = librosa.display.specshow(mel_db(y), sr=SR, hop_length=128, x_axis="time",
       ↪ y_axis="mel",
66                                    fmax=SR / 2, cmap="magma", vmin=-80, vmax=0, ax=ax)
67     ax.set_title(title, fontsize=10)
68     ax.set_xlabel("Time (s)", fontsize=9)
69     ax.set_ylabel("Frequency (Hz)", fontsize=9)
70     ax.tick_params(labelsize=8)
71     return img
```

- **Line 64.** `plot_spectrogram` draws the same dB spectrogram with axes, for figures in the report.
- **Lines 65–66.** `specshow` draws time in seconds on x and frequency on the mel scale on y, labelled in Hz up to 8 kHz. `vmin=-80, vmax=0` fixes the colour range so every labelled spectrogram uses the same colours for the same dB value.
- **Lines 67–70.** Title, axis labels and tick-label size.
- **Line 71.** Returns the drawn image, which a colour bar needs.

```
74 rows, img_paths, clips = [], [], {}
75 for i, r in df.iterrows():
76     y = load_audio(r["path"])
77     clips[i] = y
78     rows.append(get_features(y))
79     img_paths.append(save_spectrogram_image(y, spec_dir / r["file"].replace(".wav",
       ↪ ".png")))
```

The main loop applies all of the above to the 30 recordings.

- **Line 74.** Three empty containers: a list for the feature dictionaries, a list for the image paths, and a dictionary for the processed audio.
- **Line 75.** Goes through every recording; `i` is the row number (0–29) and `r` the row.
- **Line 76.** Loads and preprocesses the recording.
- **Line 77.** Keeps the processed audio in `clips` under its row number, so later figure cells do not have to load and process every file again.
- **Line 78.** Computes the 67 features and adds the dictionary to the list.
- **Line 79.** Saves the CNN image as `SPECTROGRAMS/<same name>.png` and remembers its path.

```
81 feat_df = pd.DataFrame(rows).fillna(0.0)
82 X_hand = feat_df.values
83 print("features:", X_hand.shape)
84 feat_df.assign(file=df["file"]).to_csv(out_dir / "acoustic_features.csv", index=False)
```

- **Line 81.** Turns the 30 dictionaries into a table with 30 rows and 67 columns. `fillna(0.0)` replaces any missing value with 0 as a safety net.
- **Line 82.** `X_hand` is the same numbers as a plain array, the form scikit-learn expects.
- **Line 83.** Prints the shape.
- **Line 84.** Saves the features with the filename added as the last column (`assign` adds a column without changing `feat_df`).

Expected output with your data. `features: (30, 67)`.

11 Demo: one recording to one feature vector

Code cell 11

Notebook heading: Example: one recording to a feature vector

This cell is demo part B. It repeats the preprocessing of code cell 10 step by step for a single file (Y1-014, "Salamat"), shows what each step does, lists the 67 features with their values, and proves the result is identical to that participant's row in the saved dataset. Change `demo_file` to show another recording.

Uses: `audio_dir`, `SR`, `N_MELS`, `get_features`, `feat_df`, `df`.
Creates: `steps` (the step table), `vec` (the 67 features), `vec_table`, `vec_row`.
Saves to the results folder: nothing; this cell is only for showing.

```
1 demo_file = "Y1-014_Y1_AM_Salamat.wav"
2 demo_path = audio_dir / demo_file
```

- **Line 1.** The recording to demonstrate.
- **Line 2.** Its full path.

```
4 raw, raw_sr = librosa.load(demo_path, sr=None, mono=False)
5 raw_channels = 1 if raw.ndim == 1 else raw.shape[0]
6 raw_mono = raw if raw.ndim == 1 else raw.mean(axis=0)
7 y16, _ = librosa.load(demo_path, sr=SR, mono=True)          # resample + mono
8 y_trim, (start, end) = librosa.effects.trim(y16, top_db=25)  # cut silence at the
      ↪ start/end
9 y_norm = librosa.util.normalize(y_trim)                      # peak = 1
```

- **Line 4.** Step 1: loads the file exactly as recorded, at its own sample rate and with both channels.
- **Line 5.** Number of channels (2 for these files).
- **Line 6.** A mono version of the raw file, used only to fill in the table.

- **Line 7.** Step 2: loads the file again at 16 kHz and in mono, the same call as in `load_audio`.
- **Line 8.** Step 3: trims silence at both ends with the same 25 dB rule. This time the second result is kept: `(start, end)` are the sample positions of the part that was kept, used to shade it in the plot.
- **Line 9.** Step 4: peak-normalises, so the peak becomes 1.

Note. `load_audio` also pads clips shorter than 1024 samples. That step is skipped here because this clip is far longer; the last line of the cell confirms the result is identical anyway.

```
12 def step_row(name, y, sr, ch):
13     return [name, f"{sr} Hz", ch, len(y), round(len(y) / sr, 3),
       ↪ round(float(np.abs(y).max()), 3)]
14
15
16 steps = pd.DataFrame([
17     step_row("1. raw wav", raw_mono, raw_sr, raw_channels),
18     step_row("2. 16 kHz mono", y16, SR, 1),
19     step_row("3. trimmed", y_trim, SR, 1),
20     step_row("4. normalised", y_norm, SR, 1),
21 ], columns=["step", "sample rate", "channels", "samples", "duration (s)", "peak"])
22 print(demo_file)
23 display(steps)
```

- **Lines 12–13.** A helper that describes one version of the signal in one table row: step name, sample rate, channels, number of samples, duration in seconds, and peak.
- **Lines 16–21.** The table with one row per step.
- **Lines 22–23.** Prints the filename and shows the table.

Expected output with your data. raw 192,000 Hz, 2 channels, 184,320 samples, 0.960 s, peak 0.251 → 16 kHz mono 15,360 samples, 0.960 s → trimmed 14,848 samples, 0.928 s → normalised, peak 1.000.

```
25 fig, ax = plt.subplots(3, 1, figsize=(11, 7), constrained_layout=True)
26 ax[0].plot(np.arange(len(y16)) / SR, y16, linewidth=0.6, color="#35206B")
27 ax[0].axvspan(start / SR, end / SR, color="#F9B95C", alpha=0.3, label="kept after
       ↪ trimming")
28 ax[0].set_title("16 kHz mono signal (shaded part is kept)")
29 ax[0].legend(loc="upper right")
30 ax[1].plot(np.arange(len(y_norm)) / SR, y_norm, linewidth=0.6, color="#B5367A")
31 ax[1].set_title("trimmed and normalised")
32 ax[1].set_ylim(-1.05, 1.05)
33 S = librosa.power_to_db(librosa.feature.melspectrogram(y=y_norm, sr=SR, n_fft=1024,
       ↪ hop_length=128, n_mels=N_MELS), ref=np.max)
34 librosa.display.specshow(S, sr=SR, hop_length=128, x_axis="time", y_axis="mel", fmax=SR /
       ↪ 2, cmap="magma", ax=ax[2])
35 ax[2].set_title("mel-spectrogram")
36 ax[0].set_xlabel("Time (s)")
37 ax[1].set_xlabel("Time (s)")
38 plt.show()
```

- **Line 25.** A figure with three panels stacked vertically.

- **Line 26.** Top panel: the 16 kHz signal against time. Dividing the sample positions by 16,000 converts them to seconds.
- **Line 27.** Shades the part that trimming keeps (from `start` to `end`) in light orange.
- **Lines 28–29.** Title and legend for the top panel.
- **Line 30.** Middle panel: the trimmed, normalised signal, the actual input to feature extraction.
- **Lines 31–32.** Its title, and a y-axis from −1.05 to 1.05 to show that the peak is now at 1.
- **Line 33.** The dB mel spectrogram of the processed signal (without padding to 1.5 s).
- **Line 34.** Bottom panel: draws it with time and frequency axes.
- **Lines 35–37.** Title and x-axis labels.
- **Line 38.** Shows the figure. It is not saved.

```
40 vec = get_features(y_norm)
```

- **Line 40.** Step 5: computes the 67 features of the processed signal with the same `get_features` function used for the dataset.

```
43 def feature_group(name):
44     for prefix, group in [("ddmfcc", "MFCC delta-delta SD"), ("dmfcc", "MFCC delta SD"),
       ↪ ("mfcc", "MFCC mean/SD"),
45                           ("f0", "Pitch"), ("voiced", "Pitch"), ("rms", "Energy"),
       ↪ ("zcr", "Zero-crossing rate"),
46                           ("centroid", "Spectral centroid")]:
47         if name.startswith(prefix):
48             return group
49     return "Timing"
```

- **Line 43.** A helper that names the group a feature belongs to, so the 67 features can be shown grouped.
- **Lines 44–46.** The list of name prefixes and their group. The more specific prefixes (`ddmfcc`, `dmfcc`) are checked before `mfcc`. `rms_peak_pos` starts with `rms`, so it counts as an energy feature.
- **Lines 47–48.** Returns the first group whose prefix matches.
- **Line 49.** Names that match nothing (`duration`, `n_segments`, `active_ratio`) are timing features.

```
52 vec_table = pd.DataFrame({"feature": list(vec), "value": [round(float(v), 4) for v in
       ↪ vec.values()]})
53 vec_table["group"] = vec_table["feature"].map(feature_group)
54 print(len(vec_table), "features")
55 display(vec_table.groupby("group").size().to_frame("count"))
56 pd.set_option("display.max_rows", 100)
57 display(vec_table[["group", "feature", "value"]])
```

- **Line 52.** A table with one row per feature: its name and its value rounded to 4 decimals.
- **Line 53.** Adds each feature's group.

- **Line 54.** Prints the number of features.
- **Line 55.** How many features are in each group.
- **Line 56.** Raises pandas' display limit to 100 rows, so all 67 rows are shown instead of a shortened list.
- **Line 57.** Shows the full list.

Expected output with your data. `67 features`; Energy 4, MFCC delta SD 13, MFCC delta-delta SD 13, MFCC mean/SD 26, Pitch 4, Spectral centroid 2, Timing 3, Zero-crossing rate 2. For example `duration` 0.928, `f0_mean` 199.45 Hz, `rms_mean` 0.0875.

```
59 vec_row = pd.DataFrame([vec])[feat_df.columns]
60 display(vec_row)
61 saved_row = feat_df.loc[df.index[df["file"] == demo_file][0]].values
62 print("same as the saved row:", np.allclose(vec_row.values[0], saved_row, equal_nan=True))
```

- **Line 59.** The same 67 values as one row with the columns in the same order as the dataset: the "structured feature vector".
- **Line 60.** Shows it.
- **Line 61.** Finds this participant's row in `feat_df`, the table computed for the dataset in code cell 10.
- **Line 62.** `np.allclose` checks that all 67 values are equal (allowing for tiny rounding differences in the last decimal places). `equal_nan=True` would treat two empty values as equal.

Expected output with your data. `same as the saved row: True`.

12 Labelled spectrogram figures

Code cell 12

This cell draws the labelled spectrograms used in the report: one summary figure with 8 recordings (Figure 1 in the report), and one separate labelled figure for each of the 30 recordings.

Uses: `df`, `clips`, `plot_spectrogram`.

Saves to the results folder: `spectrogram_examples.png` (or `spectrogram_examples_<word>.png`) and 30 PNG files in `spectrograms_labelled/`.

```
1 # spectrograms of the first 8 recordings (Figure 1 in the report)
2 # set show_word = "Kapoy" to compare only one word instead
3 n_show = 8
4 show_word = None
```

- **Lines 1–2.** Comments explaining the two settings below.
- **Line 3.** How many recordings to show in the summary figure.
- **Line 4.** `None` shows the first recordings in the table. Setting it to a word, for example `"Kapoy"`, shows only that word, sorted from lowest to highest valence.

```
6  show = df.index.tolist() if show_word is None else df[df["word"] ==
       ↪ show_word].sort_values("valence").index.tolist()
7  show = show[:n_show]
```

- **Line 6.** The row numbers to show: either all rows, or only the rows with that word sorted by valence.
- **Line 7.** Keeps only the first 8.

Note. If `show_word` is set to a word that does not exist, the list is empty, nothing is drawn, and the colour bar on line 18 fails with an error. Use one of the eight words exactly as spelled.

```
9  ncols = 4
10 nrows = math.ceil(len(show) / ncols)
11 fig, axes = plt.subplots(nrows, ncols, figsize=(4.4 * ncols, 3.5 * nrows), squeeze=False,
       ↪ constrained_layout=True)
```

- **Line 9.** Four panels per row.
- **Line 10.** Number of rows, rounded up: $8 \div 4 = 2$.
- **Line 11.** Creates the grid of panels, 4.4 inches wide and 3.5 inches tall per panel. `squeeze=False` keeps the panels in a two-dimensional grid even when there is only one row; `constrained_layout=True` spaces them automatically.

```
12 for a in axes.ravel():
13     a.set_visible(False)
14 for a, i in zip(axes.ravel(), show):
15     a.set_visible(True)
16     r = df.loc[i]
17     img = plot_spectrogram(a, clips[i], f"{r['word']} | V={r['valence']}
       ↪ A={r['arousal']}")
```

- **Lines 12–13.** Hides every panel first, so unused panels stay empty.
- **Line 14.** Pairs the panels (`ravel()` lists them row by row) with the recordings to show.
- **Line 15.** Makes this panel visible again.
- **Line 16.** The recording's row.
- **Line 17.** Draws its spectrogram with a title such as "Kapoy | V=4 A=4".

```
18 fig.colorbar(img, ax=axes.ravel().tolist(), shrink=0.85, pad=0.02,
19              label="Power (dB, relative to each clip's peak)", format="%+.0f dB")
20 fig.suptitle("Mel-spectrograms of example recordings (V = valence, A = arousal)",
       ↪ fontsize=12)
21 save_fig(fig, "spectrogram_examples" if show_word is None else
       ↪ f"spectrogram_examples_{show_word.lower()}")
22 plt.show()
```

- **Lines 18–19.** One colour bar shared by all panels, labelled in dB relative to each clip's peak.

- **Line 20.** The overall title.
- **Line 21.** Saves the figure; the name includes the word if `show_word` was set.
- **Line 22.** Shows it.

```
24 # one labelled spectrogram per recording
25 for i, r in df.iterrows():
26     name = r["file"].replace(".wav", "")
27     fig, a = plt.subplots(figsize=(6.5, 4.2), constrained_layout=True)
28     img = plot_spectrogram(a, clips[i], f"{r['word']} | V={r['valence']}
       ↪ A={r['arousal']}\n{name}")
29     fig.colorbar(img, ax=a, label="Power (dB, relative to clip's peak)", format="%+.0f
       ↪ dB")
30     fig.savefig(spec_labelled_dir / f"{name}.png", dpi=200, bbox_inches="tight")
31     plt.close(fig)
32 print("saved", len(df), "labelled spectrograms")
```

- **Line 24.** A comment for the second part.
- **Line 25.** Goes through all 30 recordings.
- **Line 26.** The filename without `.wav`, used in the title and the file name.
- **Line 27.** A new figure with one panel.
- **Line 28.** Draws the spectrogram, with the word, ratings and filename in the title.
- **Line 29.** Adds the colour bar.
- **Line 30.** Saves it in `spectrograms_labelled/` at 200 dpi.
- **Line 31.** Closes the figure, so 30 figures are not kept in memory or printed in the notebook.
- **Line 32.** Prints a confirmation.

13 Waveforms and clearer spectrograms

Code cell 13

Notebook heading: Waveforms and clearer spectrograms

This cell makes the clearest figures for the slides and the report. They differ from the CNN images in three ways: they show only the spoken word (no padding), they use finer time steps (4 ms instead of 8 ms), and they overlay the pitch line. The cell only draws figures; it does not change any feature or model.

Uses: `df`, `clips`, `out_dir`, `SR`.

Creates: the functions `plot_wave` and `plot_spec_pitch`.

Saves to the results folder: 30 figures in `waveforms_spectrograms/` and one `word_gallery_<word>.png` per word.

```
1 wave_dir = out_dir / "waveforms_spectrograms"
2 wave_dir.mkdir(parents=True, exist_ok=True)
```

```
3  HOP = 64      # 4 ms steps
4  FMAX = 8000
```

- **Lines 1–2.** The folder for the per-recording figures, created if needed.
- **Line 3.** `HOP = 64` samples, a new frame every 4 ms, which gives smooth curves.
- **Line 4.** Frequencies are shown up to 8 kHz.

```
7  def plot_wave(ax, y, xlabel=True):
8      t = np.arange(len(y)) / SR
9      ax.plot(t, y, color="#35206B", linewidth=0.6)
10     rms = librosa.feature.rms(y=y, frame_length=512, hop_length=HOP)[0]
11     t_rms = librosa.times_like(rms, sr=SR, hop_length=HOP)
12     ax.plot(t_rms, rms, color="#E8683A", linewidth=1.6, label="RMS energy")
13     ax.plot(t_rms, -rms, color="#E8683A", linewidth=1.6)
14     for s, e in librosa.effects.split(y, top_db=30):   # shade the speech parts
15         ax.axvspan(s / SR, e / SR, color="#F9B95C", alpha=0.18, linewidth=0)
16     ax.set_xlim(0, len(y) / SR)
17     ax.set_ylim(-1.05, 1.05)
18     ax.set_ylabel("Amplitude\n(peak-normalised)", fontsize=9)
19     if xlabel:
20         ax.set_xlabel("Time (s)", fontsize=9)
21     ax.tick_params(labelsize=8)
```

`plot_wave` draws the waveform panel.

- **Line 7.** Defines the function; `xlabel` controls whether the time label is drawn (only the bottom panel of a stack needs it).
- **Line 8.** The time of every sample in seconds.
- **Line 9.** Draws the waveform in dark purple with a thin line.
- **Line 10.** RMS energy with short frames (512 samples, 32 ms) every 4 ms, a detailed loudness curve.
- **Line 11.** The time of each RMS frame.
- **Lines 12–13.** Draws the energy curve above and, mirrored, below the waveform in orange: the loudness envelope.
- **Lines 14–15.** Shades each non-silent stretch (more than 30 dB below the peak counts as silence) in light yellow, showing what the timing features count as speech.
- **Lines 16–17.** The x-axis covers exactly the clip; the y-axis runs from −1.05 to 1.05.
- **Line 18.** The y-axis label.
- **Lines 19–20.** The x-axis label, only if requested.
- **Line 21.** Smaller tick labels.

```
24 def plot_spec_pitch(ax, y, xlabel=True, pitch=True):
25     S = librosa.power_to_db(librosa.feature.melspectrogram(y=y, sr=SR, n_fft=512,
```

```
        ↪ hop_length=HOP, n_mels=128, fmax=FMAX), ref=np.max)
26      img = librosa.display.specshow(S, sr=SR, hop_length=HOP, x_axis="time", y_axis="mel",
27                                     fmax=FMAX, cmap="magma", vmin=-70, vmax=0, ax=ax)
28      if pitch:
29          f0, _, _ = librosa.pyin(y, fmin=65, fmax=500, sr=SR, frame_length=1024,
        ↪ hop_length=HOP)
30          ax.plot(librosa.times_like(f0, sr=SR, hop_length=HOP), f0, color="cyan",
        ↪ linewidth=2, label="Pitch (f0)")
31          ax.legend(loc="upper right", fontsize=8, framealpha=0.6)
32      ax.grid(False)
33      ax.set_xlim(0, len(y) / SR)
34      ax.set_ylabel("Frequency (Hz)", fontsize=9)
35      ax.set_xlabel("Time (s)" if xlabel else "", fontsize=9)
36      ax.tick_params(labelsize=8)
37      return img
```

`plot_spec_pitch` draws the spectrogram panel with the pitch line.

- **Line 24.** Defines the function; `pitch` controls whether the pitch line is drawn.
- **Line 25.** A dB mel spectrogram with 32 ms windows (`n_fft=512`) every 4 ms, 128 bands up to 8 kHz, relative to the clip's peak.
- **Lines 26–27.** Draws it with time and frequency axes. The colours run from −70 dB (black) to 0 dB (pale yellow).
- **Line 28.** If the pitch line is wanted:
- **Line 29.** Estimates the pitch every 4 ms with the same 65–500 Hz range as the features.
- **Line 30.** Draws it as a cyan line over the spectrogram, at the height of its frequency. The line has gaps where there is no voice.
- **Line 31.** A small legend.
- **Line 32.** Turns off the grid lines, which would otherwise be drawn over the image.
- **Lines 33–36.** Axis limits, labels and tick size.
- **Line 37.** Returns the image for the colour bar.

```
40  # one figure per recording, only the first one is shown
41  for n, (i, r) in enumerate(df.iterrows()):
42      fig, (a1, a2) = plt.subplots(2, 1, figsize=(8, 6), sharex=True,
        ↪ gridspec_kw={"height_ratios": [1, 2.2]}, constrained_layout=True)
43      plot_wave(a1, clips[i], xlabel=False)
44      a1.legend(loc="upper right", fontsize=8)
45      img = plot_spec_pitch(a2, clips[i])
46      fig.colorbar(img, ax=a2, format="%+.0f dB", label="Power (dB, relative to clip's
        ↪ peak)")
47      fig.suptitle(f"\"{r['word']}\"  |  valence {r['valence']} ({r['valence_class']}), "
48                   f"arousal {r['arousal']}
        ↪ ({r['arousal_class']})\n{r['file'].replace('.wav', '')}", fontsize=11)
49      fig.savefig(wave_dir / r["file"].replace(".wav", ".png"), dpi=200,
        ↪ bbox_inches="tight")
50      if n == 0:
```

```
        plt.show()
    else:
        plt.close(fig)
print("saved", len(df), "waveform figures")
```

One figure per recording.

- **Line 40.** A comment.
- **Line 41.** Goes through every recording; `n` counts 0, 1, 2, ... so the first figure can be treated differently.
- **Line 42.** Two panels stacked, sharing the time axis; the spectrogram is 2.2 times taller than the waveform.
- **Lines 43–44.** The waveform on top, without a time label, and its legend.
- **Line 45.** The spectrogram with pitch underneath.
- **Line 46.** The colour bar.
- **Lines 47–48.** A two-line title: the word, both ratings with their classes, and the filename.
- **Line 49.** Saves it in `waveforms_spectrograms/` under the recording's name.
- **Lines 50–53.** Shows only the first figure in the notebook as an example and closes the others.
- **Line 54.** Prints a confirmation.

```
# one gallery per word, same time axis in every row
for word, g in df.groupby("word"):
    g = g.sort_values(["valence", "arousal"])
    longest = max(len(clips[i]) for i in g.index) / SR
    fig, axes = plt.subplots(len(g), 2, figsize=(13, 2.1 * len(g) + 0.8), squeeze=False,
                             gridspec_kw={"width_ratios": [1, 1.25]},
    ↪ constrained_layout=True)
    for row, (i, r) in enumerate(g.iterrows()):
        last = row == len(g) - 1
        plot_wave(axes[row, 0], clips[i], xlabel=last)
        img = plot_spec_pitch(axes[row, 1], clips[i], xlabel=last)
        axes[row, 0].set_xlim(0, longest)
        axes[row, 1].set_xlim(0, longest)
        axes[row, 1].get_legend().remove()
        axes[row, 0].set_title(f"{r['participant']}  |  V={r['valence']}
    ↪ A={r['arousal']}", fontsize=9, loc="left")
    fig.colorbar(img, ax=axes[:, 1].tolist(), format="%+.0f dB", shrink=0.6, label="dB
    ↪ re. clip peak")
    fig.suptitle(f"\"{word}\" — {len(g)} recording(s), sorted by valence (cyan line =
    ↪ pitch, shaded = speech)", fontsize=12)
    fig.savefig(out_dir / f"word_gallery_{word.lower()}.png", dpi=200,
    ↪ bbox_inches="tight")
    plt.close(fig)
print("saved the word galleries")
```

One gallery per word, so recordings of the same word can be compared.

- **Line 56.** A comment.

- **Line 57.** `groupby("word")` gives each word with its recordings (`g`).
- **Line 58.** Sorts them by valence, then arousal.
- **Line 59.** The duration of the longest recording of this word, so every row can use the same time axis.
- **Lines 60–61.** One row per recording and two columns (waveform, spectrogram); the figure height grows with the number of recordings.
- **Line 62.** Goes through the recordings; `row` counts the rows.
- **Line 63.** True for the last row, the only one that gets time labels.
- **Line 64.** The waveform in the left column.
- **Line 65.** The spectrogram with pitch in the right column.
- **Lines 66–67.** Gives both panels the same time axis, the longest clip of the word.
- **Line 68.** Removes the pitch legend from each row to reduce clutter (the title explains the cyan line).
- **Line 69.** Labels the row with the participant code and both ratings.
- **Line 70.** One colour bar for the right-hand column.
- **Line 71.** The overall title.
- **Lines 72–73.** Saves the gallery and closes it.
- **Line 74.** Prints a confirmation.

Note. The pitch tracker is slow, so this cell takes about 30 seconds.

14 RQ3: do the features differ between classes?

Code cells 14, 15

Notebook heading: RQ3: do the acoustic features differ between classes?

These two cells compare the acoustic features of the two classes, for valence and for arousal. Code cell 14 compares six key features with medians, means, a statistical test and boxplots. Code cell 15 gives summary statistics for every feature, and for 12 interpretable features by class.

Uses: `feat_df`, `df`.

Saves to the results folder: `rq3_<target>_acoustic_comparison.csv` and `rq3_<target>_acoustic_boxplots.png` for both targets, `audio_descriptive_statistics.csv`, `audio_descriptive_by_<target>.csv`.

Code cell 14

```
1  key_feats = ["f0_mean", "f0_std", "rms_mean", "zcr_mean", "duration", "active_ratio"]
```

- **Line 1.** The six features to compare: average pitch, pitch variation, average energy, average zero-crossing rate, duration and active ratio.

```
3  for target in ["valence_class", "arousal_class"]:
4      d = feat_df[key_feats].copy()
5      d[target] = df[target].values
6      classes = sorted(d[target].unique())
7      rows = []
```

- **Line 3.** Repeats everything below for valence and for arousal.
- **Line 4.** A copy of those six columns.
- **Line 5.** Adds the class label as a seventh column.
- **Line 6.** The two class names in alphabetical order: non-positive and positive, or high and low-mod.
- **Line 7.** An empty list for the result rows.

```
8      for feat in key_feats:
9          a = d.loc[d[target] == classes[0], feat].values
10         b = d.loc[d[target] == classes[1], feat].values
11         row = {"feature": feat}
12         for c in classes:
13             row[f"{c} median"] = round(d.loc[d[target] == c, feat].median(), 3)
14         for c in classes:
15             vals = d.loc[d[target] == c, feat]
16             row[f"{c} mean (sd)"] = f"{vals.mean():.3f} ({vals.std():.3f})"
17         row["p (uncorrected)"] = round(mannwhitneyu(a, b).pvalue, 3)
18         rows.append(row)
```

- **Line 8.** Goes through the six features.
- **Lines 9–10.** The feature's values in the first class (`a`) and the second class (`b`).
- **Line 11.** Starts a result row with the feature's name.
- **Lines 12–13.** Adds the median of each class (the middle value, which is not pulled around by one extreme recording).
- **Lines 14–16.** Adds each class's mean and standard deviation as text such as "0.122 (0.034)". pandas' `.std()` divides by $N - 1$ (sample standard deviation).
- **Line 17.** The Mann–Whitney U test: it ranks all values of both classes together and checks whether one class's values tend to rank higher. It does not assume normally distributed values, which suits small samples. The two-sided p-value is rounded to 3 decimals.
- **Line 18.** Adds the row.

Note. The p-values are not corrected for multiple testing. Six features × two targets = 12 tests; at the 0.05 level, about 0.6 "significant" results are expected by chance alone. With a Bonferroni correction, a p-value would need to be below $0.05 \div 12 \approx 0.004$. No feature came close (the smallest was 0.072, energy for valence).

```
19     tbl = pd.DataFrame(rows)
20     print("\n" + target, d[target].value_counts().to_dict())
```

```
21      display(tbl)
22      tbl.to_csv(out_dir / f"rq3_{target}_acoustic_comparison.csv", index=False,
        ↪ encoding="utf-8")
```

- **Line 19.** The six result rows as a table.
- **Line 20.** Prints the target name and its class sizes.
- **Lines 21–22.** Shows and saves the table.

```
24      fig, ax = plt.subplots(1, len(key_feats), figsize=(16, 3.4))
25      for a, feat in zip(ax, key_feats):
26          sns.boxplot(data=d, x=target, y=feat, ax=a, color="lightgray")
27          sns.stripplot(data=d, x=target, y=feat, ax=a, color="black", size=4)
28          a.set_xlabel("")
29          a.set_title(feat, fontsize=9)
30      fig.suptitle(f"Acoustic features by {target.replace('_', ' ')} (each dot = one
        ↪ recording)", fontsize=11)
31      plt.tight_layout()
32      save_fig(fig, f"rq3_{target}_acoustic_boxplots")
33      plt.show()
```

- **Line 24.** Six panels side by side, one per feature.
- **Line 25.** Pairs each panel with its feature.
- **Line 26.** A boxplot per class: the box spans the middle half of the values, the line inside is the median, and the whiskers show the range without outliers.
- **Line 27.** The individual recordings as black dots on top, so every one of the 30 values is visible.
- **Lines 28–29.** Removes the x-axis label (the class names are already on the axis) and titles the panel with the feature name.
- **Line 30.** The overall title.
- **Lines 31–33.** Spacing, save, show.

Expected output with your data. valence, non-positive vs positive medians: `rms_mean` 0.114 vs 0.095 ($p = 0.072$), `duration` 0.80 s vs 0.94 s ($p = 0.156$), `f0_mean` 204 vs 201 Hz ($p = 0.816$). For arousal every p-value is above 0.14.

Code cell 15

```
1  # descriptive stats for all features, then a few of them per class
2  desc = feat_df.describe().T.round(3)
3  display(desc)
4  desc.to_csv(out_dir / "audio_descriptive_statistics.csv", encoding="utf-8")
```

- **Line 1.** A comment.
- **Line 2.** `describe()` for all 67 features, transposed with `.T` so each feature is a row with its count, mean, standard deviation, minimum, quartiles and maximum.
- **Lines 3–4.** Shows and saves it.

```
6  some_feats = ["duration", "f0_mean", "f0_std", "f0_range", "voiced_ratio", "rms_mean",
       ↪ "rms_max",
7                "zcr_mean", "centroid_mean", "n_segments", "active_ratio", "rms_peak_pos"]
8  for target in ["valence_class", "arousal_class"]:
9      by_class = feat_df[some_feats].groupby(df[target].values).agg(["mean",
       ↪ "std"]).round(3).T
10     print("\n" + target)
11     display(by_class)
12     by_class.to_csv(out_dir / f"audio_descriptive_by_{target}.csv", encoding="utf-8")
```

- **Lines 6–7.** Twelve features that are easy to interpret (the timing, pitch, energy, ZCR and centroid features, without the 52 MFCC columns).
- **Line 8.** For valence and for arousal:
- **Line 9.** Groups the 30 rows by class and computes the mean and standard deviation of each feature, then transposes so the features are rows.
- **Lines 10–12.** Prints the target name, shows and saves the table.

15 Spectrogram embeddings with ResNet18

Code cell 16

Notebook heading: Spectrogram embeddings

This cell turns each of the 30 spectrogram images into 512 numbers with ResNet18, a convolutional neural network pretrained on ImageNet (about 1.2 million everyday photos in 1000 categories). The network is used only as a fixed feature extractor: it is not trained or fine-tuned on the recordings. These 512 numbers are the second feature set compared in code cell 18.

Uses: `img_paths`.
Creates: `net` (the network), `X_cnn` (30 × 512 array).
Saves to the results folder: `cnn_embeddings.npy`.

```
1  net = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
2  net.fc = torch.nn.Identity()
3  net.eval()
4  to_tensor = T.Compose([T.ToTensor(), T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
       ↪ 0.224, 0.225])])
```

- **Line 1.** Loads ResNet18 with its pretrained ImageNet weights. The first time, this downloads a file of about 45 MB, so it needs an internet connection.
- **Line 2.** The last layer of ResNet18 (`fc`) turns 512 internal values into scores for the 1000 ImageNet classes. Replacing it with `Identity` (a layer that passes its input through unchanged) makes the network output the 512 values themselves, the result of its final pooling layer.
- **Line 3.** Puts the network in evaluation mode: its batch-normalisation layers use the statistics stored from training instead of updating them.
- **Line 4.** The preparation every image needs: `ToTensor` converts the 224 × 224 colour image into a 3 × 224 × 224 array of values between 0 and 1; `Normalize` subtracts the average of each colour

channel in ImageNet and divides by its standard deviation, the same preparation the network was trained with.

```
6  embeddings = []
7  with torch.no_grad():
8      for p in img_paths:
9          x = to_tensor(Image.open(p).convert("RGB")).unsqueeze(0)
10         embeddings.append(net(x).squeeze(0).numpy())
```

- **Line 6.** An empty list for the 30 results.
- **Line 7.** `torch.no_grad()` turns off the bookkeeping needed for training, which saves memory and time.
- **Line 8.** Goes through the 30 image files.
- **Line 9.** Opens the image, makes sure it has three colour channels, applies the preparation, and adds a batch dimension with `unsqueeze(0)` (the network expects a batch of images, here a batch of one): shape $1 \times 3 \times 224 \times 224$.
- **Line 10.** Runs the network, removes the batch dimension again, converts the result to a NumPy array of 512 values, and adds it to the list.

```
11 X_cnn = np.vstack(embeddings)
12 print("CNN embeddings:", X_cnn.shape)
13 np.save(out_dir / "cnn_embeddings.npy", X_cnn)
```

- **Line 11.** Stacks the 30 arrays into one array with 30 rows and 512 columns.
- **Line 12.** Prints the shape.
- **Line 13.** Saves it in NumPy's own file format.

Expected output with your data. `CNN embeddings: (30, 512).`

Note. ImageNet features describe shapes and textures in photos, not speech. That they did not beat the handcrafted features (valence macro-F1 0.61 vs 0.72) is not surprising. Fine-tuning the network on spectrograms would need far more than 30 recordings.

16 Saving the dataset

Code cell 17

Notebook heading: Saving the dataset

This cell writes the final dataset: one row per recording, with the context and labels first and the 67 audio features after them. It also writes a data dictionary that says which group every column belongs to.

Uses: `df`, `meta_by_file`, `feat_df`.
Creates: `info`, `audio_cols`, `dataset` (30×78), `dictionary`.
Saves to the results folder: `datasets/Group1_audio_feature_dataset.csv` and `datasets/data_`

dictionary.csv.

```
info = pd.DataFrame({
    "participant_code": df["participant"].values,
    "audio_filename": df["file"].values,
    "year_level": df["file"].map(meta_by_file["Year Level"]).values,
    "class_activity": df["file"].map(meta_by_file["Class Activity"]).values,
    "session": df["file"].map(meta_by_file["Session"]).values,
    "spoken_word": df["word"].values,
    "self_reported_feeling": df["file"].map(meta_by_file["Self-Reported Feeling"]).values,
    "valence": df["valence"].values,
    "arousal": df["arousal"].values,
    "valence_class": df["valence_class"].values,
    "arousal_class": df["arousal_class"].values,
})
```

- **Lines 1–13.** The 11 context and label columns, with short lower-case names: participant code, audio filename, year level, class activity and session (looked up from the metadata by filename), spoken word, the free-text feeling, the two ratings and the two classes. These columns describe the data but are never used as model inputs.

```
audio_cols = feat_df.add_prefix("aud_").reset_index(drop=True)
dataset = pd.concat([info, audio_cols], axis=1)
dataset.to_csv(data_dir / "Group1_audio_feature_dataset.csv", index=False,
    ↪ encoding="utf-8")
```

- **Line 14.** The 67 features with `aud_` added in front of every name (for example `aud_duration`), so they can be told apart from the context columns. `reset_index(drop=True)` makes the row numbers 0–29 so the two tables line up.
- **Line 15.** Puts the two tables side by side (`axis=1`): 11 + 67 = 78 columns.
- **Line 16.** Saves the dataset.

```
dictionary = pd.DataFrame(
    [{"column": c, "group": "metadata (context and labels, NOT predictors)"} for c in
    ↪ info.columns]
    + [{"column": c, "group": "audio feature (handcrafted)"} for c in audio_cols.columns])
dictionary.to_csv(data_dir / "data_dictionary.csv", index=False, encoding="utf-8")
```

- **Lines 18–20.** A two-column table listing every column and its group: “metadata (context and labels, NOT predictors)” or “audio feature (handcrafted)”. The two lists are joined with `+`.
- **Line 21.** Saves it.

```
print("dataset:", dataset.shape)
display(dataset.iloc[:3, :14])
```

- **Line 23.** Prints the shape.
- **Line 24.** Shows the first 3 rows and the first 14 columns.

Expected output with your data. `dataset: (30, 78)`.

17 Training and testing the models (RQ4, RQ5)

Code cell 18

Notebook heading: Models (RQ4, RQ5)

This cell is the machine-learning experiment and the start of demo part C. It compares three classifiers on two feature sets, plus a majority-class baseline, for both valence and arousal, using leave-one-participant-out evaluation.

Leave-one-participant-out (LOPO). The 30 participants give 30 rounds ("folds"). In each round the model is trained on 29 people and predicts the one left out. After 30 rounds every participant has exactly one prediction made by a model that never saw them. This is the strictest test possible with 30 people, and it is what the project sheet asks for: speaker-independent evaluation.

The pipeline. Each model is three steps trained together: scale every feature to mean 0 and standard deviation 1, reduce to 5 principal components, then classify. Because the whole pipeline is trained inside each round, the scaling and PCA never see the person being tested.

Uses: `X_hand`, `X_cnn`, `df`, `N_PCA`, `SEED`.

Creates: `feature_sets`, `model_names`, `TARGETS`, `folds` (the 30 rounds), the functions `make_model` and `run_lopo`, `configs` (7 configurations), `results` (score tables), `preds` (all predictions).

Saves to the results folder: `model_results.csv`.

```
1  feature_sets = {"Audio: handcrafted": X_hand, "Audio: spectrogram CNN": X_cnn}
2  model_names = ["LogReg", "SVM", "RandomForest"]
3  TARGETS = ["valence_class", "arousal_class"]
```

- **Line 1.** The two feature sets: the 67 handcrafted features and the 512 CNN values.
- **Line 2.** The three classifiers.
- **Line 3.** The two things to predict.

```
5  folds = list(LeaveOneGroupOut().split(X_hand, groups=df["participant"].values))
6  print(len(folds), "folds")
```

- **Line 5.** Builds the folds. `LeaveOneGroupOut` uses `groups` (the participant codes): each fold tests one participant and trains on all others. `list(...)` stores the 30 pairs of (training rows, test row) so exactly the same folds are reused in every later cell.
- **Line 6.** Prints the number of folds.

Note. Since each participant has exactly one recording, this is the same as leaving out one recording at a time. Grouping by participant makes the rule explicit, and it would stay correct if participants had several recordings each.

```
9  def make_model(name):
10     if name == "LogReg":
11         clf = LogisticRegression(C=0.5, max_iter=2000, class_weight="balanced")
12     elif name == "SVM":
13         clf = SVC(kernel="rbf", C=1.0, class_weight="balanced")
14     else:
15         clf = RandomForestClassifier(n_estimators=100, max_depth=4, min_samples_leaf=2,
16                                      class_weight="balanced", random_state=SEED)
17     return make_pipeline(StandardScaler(), PCA(n_components=N_PCA, random_state=SEED),
       ↪ clf)
```

- **Line 9.** `make_model` builds one untrained pipeline for the given classifier name.
- **Lines 10–11. Logistic regression**: a linear model that converts a weighted sum of the inputs into a probability with the logistic function. `C=0.5` sets the regularisation: a smaller `C` keeps the weights smaller and the model simpler, which matters with 29 training examples. `max_iter=2000` gives the solver enough steps to converge.
- **Lines 12–13. Support vector machine** with an RBF kernel: it can draw a curved boundary between the classes. `C=1.0` is the default trade-off between a smooth boundary and fitting every training example; the kernel width uses scikit-learn's default (`gamma="scale"`).
- **Lines 14–16. Random forest**: 100 decision trees, each trained on a random sample of the data, voting together. `max_depth=4` and `min_samples_leaf=2` keep each tree small to limit overfitting; `random_state=SEED` makes the random sampling repeatable.
- **Line 17.** Chains `StandardScaler` → `PCA(5)` → the classifier. The PCA `random_state` matters for the 512 CNN columns: with more than 500 columns, scikit-learn chooses a randomized algorithm, and the seed keeps its result the same every run.

All three classifiers use `class_weight="balanced"`: mistakes on the smaller class count more, so a model cannot score well by always predicting the larger class.

Formula. standardising feature j: $z_j = (x_j - \mu_j)/\sigma_j$, with μ_j, σ_j from the 29 training people. Balanced class weight for class c with n_c of the n training examples and $K = 2$ classes: $w_c = \frac{n}{K\, n_c}$, for example $30/(2 \times 18) = 0.83$ for non-positive and $30/(2 \times 12) = 1.25$ for positive. Logistic regression: $P(\text{class} = 1) = 1/(1 + e^{-(b + w_1 z_1 + \cdots + w_5 z_5)})$.

```
20 def run_lopo(config, y):
21     if config["model"] == "Baseline":
22         return cross_val_predict(DummyClassifier(strategy="most_frequent"),
       ↪ np.zeros((len(y), 1)), y, cv=folds)
23     return cross_val_predict(make_model(config["model"]),
       ↪ feature_sets[config["features"]], y, cv=folds)
```

`run_lopo` runs the full leave-one-participant-out evaluation for one configuration and returns 30 predictions.

- **Line 20.** Defines the function; `config` says which feature set and model, `y` holds the 30 labels.
- **Lines 21–22. The baseline.** `DummyClassifier(strategy="most_frequent")` ignores its input completely and always predicts the most common class among the 29 training people. It is given a column of zeros as input, because it uses no features. Running it through the same

folds gives the accuracy a model would get with no information from the audio: 18/30 for valence (always "non-positive") and 20/30 for arousal (always "low-mod").

- **Line 23.** For a real model: `cross_val_predict` takes a fresh copy of the pipeline for each of the 30 folds, trains it on the 29 training people, predicts the left-out person, and returns the 30 predictions in the original row order.

```
26 configs = [{"features": "Baseline (majority)", "model": "Baseline"}]
27 for fs in feature_sets:
28     for m in model_names:
29         configs.append({"features": fs, "model": m})
```

- **Line 26.** The list of configurations starts with the baseline.
- **Lines 27–29.** Adds every combination of feature set and model: $2 \times 3 = 6$. The position in the list is the configuration number used later (`cfg_id`): 0 baseline; 1–3 handcrafted with logistic regression, SVM, random forest; 4–6 CNN with the same three.

```
31 results, preds = {}, {}
32 for target in TARGETS:
33     y = df[target].values
34     rows = []
35     for k, config in enumerate(configs):
36         p = run_lopo(config, y)
37         preds[(target, k)] = p
38         rows.append({"features": config["features"], "model": config["model"],
39                      "accuracy": round(accuracy_score(y, p), 3),
40                      "macro_f1": round(f1_score(y, p, average="macro"), 3), "cfg_id": k})
41     results[target] = pd.DataFrame(rows).sort_values("macro_f1",
       ↪ ascending=False).reset_index(drop=True)
42     print("\n" + target)
43     display(results[target].drop(columns="cfg_id"))
```

- **Line 31.** Two empty dictionaries: score tables and predictions.
- **Line 32.** For valence and for arousal:
- **Line 33.** The 30 true labels.
- **Line 34.** An empty list of result rows.
- **Line 35.** For each of the 7 configurations, with its number `k`:
- **Line 36.** Runs the 30 folds.
- **Line 37.** Stores the predictions under (target, configuration number) so later cells can reuse them without retraining.
- **Lines 38–40.** Records accuracy and macro-F1, rounded to 3 decimals, and the configuration number.
- **Line 41.** The 7 rows as a table, sorted from highest to lowest macro-F1.
- **Lines 42–43.** Prints the target and shows the table without the configuration-number column.

Formula. accuracy $= \frac{\text{correct}}{30}$. For each class, precision $P = \frac{TP}{TP+FP}$, recall $R = \frac{TP}{TP+FN}$, $F_1 = \frac{2PR}{P+R}$. Macro-F1 is the average of the two classes' F_1, so the smaller class counts as much as the larger one.

```
45 all_results = pd.concat([r.assign(target=t) for t, r in
       ↪ results.items()]).drop(columns="cfg_id")
46 all_results.to_csv(out_dir / "model_results.csv", index=False)
```

- **Line 45.** Joins the valence and arousal tables with a column saying which target each row belongs to.
- **Line 46.** Saves them.

Expected output with your data. `30 folds`. Valence (accuracy / macro-F1): handcrafted + LogReg 0.733 / 0.722, handcrafted + SVM 0.700 / 0.683, CNN + LogReg 0.633 / 0.612, CNN + SVM 0.533 / 0.514, handcrafted + RF 0.500 / 0.486, CNN + RF 0.433 / 0.377, baseline 0.600 / 0.375. Arousal: CNN + LogReg 0.600 / 0.550 is the highest; the baseline has 0.667 accuracy.

Note. Choosing the best of six configurations using the same 30 people that score them makes the best score optimistic. A separate validation set would avoid this, but 30 people are too few to split further.

18 Confusion matrices and per-class results

Code cell 19

Notebook heading: Confusion matrices

This cell takes the highest-scoring configuration for each target, draws its confusion matrix, prints precision, recall and F1 per class, and saves every recording's prediction so the errors can be analysed by word.

Uses: `results`, `preds`, `configs`, `df`, `TARGETS`.

Creates: the function `top_model`, `best_cfg` (the chosen configuration per target, reused by code cells 21 and 22), `pred_df`.

Saves to the results folder: `confusion_matrices.png`, `best_model_predictions.csv`.

```
1 def top_model(target):
2     r = results[target]
3     return r[r["model"] != "Baseline"].iloc[0]
```

- **Lines 1–3.** `top_model` returns the first row of the sorted results table that is not the baseline, which is the configuration with the highest macro-F1.

```
6 best_cfg = {}
7 pred_rows = []
8 fig, ax = plt.subplots(2, 1, figsize=(5.5, 9))
```

- **Line 6.** An empty dictionary for the chosen configuration per target.
- **Line 7.** An empty list for per-recording predictions.

- **Line 8.** A figure with two panels stacked vertically.

```
9  for a, target in zip(ax, TARGETS):
10     y = df[target].values
11     labels = sorted(set(y))
12     best = top_model(target)
13     k = int(best["cfg_id"])
14     best_cfg[target] = configs[k]
15     p = preds[(target, k)]
```

- **Line 9.** Pairs each panel with a target.
- **Line 10.** The true labels.
- **Line 11.** The two class names in alphabetical order, so rows and columns of the matrix are always in the same order.
- **Line 12.** The highest-scoring configuration.
- **Line 13.** Its configuration number.
- **Line 14.** Remembers the configuration itself for later cells.
- **Line 15.** Its 30 stored predictions.

```
17     sns.heatmap(confusion_matrix(y, p, labels=labels), annot=True, fmt="d", cmap="Blues",
18                 xticklabels=labels, yticklabels=labels, ax=a, cbar=False)
19     a.set_title(f"{target}\n{best['features']} ({best['model']})", fontsize=10)
20     a.set_xlabel("Predicted")
21     a.set_ylabel("Self-reported")
22     print(f"\n{target} - {best['features']} ({best['model']})")
23     print(classification_report(y, p, zero_division=0))
```

- **Lines 17–18.** `confusion_matrix` counts, for each true class (rows), how many were predicted as each class (columns). The heatmap writes the four counts in the squares. The diagonal is correct predictions.
- **Line 19.** Title: the target and the configuration.
- **Lines 20–21.** Axis labels: predicted on x, self-reported (true) on y.
- **Line 22.** Prints the same heading.
- **Line 23.** `classification_report` prints precision, recall, F1 and the number of people (support) for each class, plus accuracy, the macro average and the weighted average. `zero_division=0` reports 0 instead of a warning if a class is never predicted.

```
25     for i in range(len(y)):
26         pred_rows.append({"target": target, "file": df.loc[i, "file"], "word": df.loc[i,
       ↪ "word"],
27                           "self_reported": y[i], "predicted": p[i], "correct": y[i] ==
       ↪ p[i]})
```

- **Lines 25–27.** For every recording, records the target, filename, word, true class, predicted class, and whether they match.

```
28 plt.tight_layout()
29 save_fig(fig, "confusion_matrices")
30 plt.show()
```

- **Lines 28–30.** Spacing, save, show.

```
32 pred_df = pd.DataFrame(pred_rows)
33 pred_df.to_csv(out_dir / "best_model_predictions.csv", index=False, encoding="utf-8")
34 print("wrong predictions per word:")
35 pred_df["wrong"] = ~pred_df["correct"]
36 display(pred_df.pivot_table(index="word", columns="target", values="wrong",
       ↪ aggfunc="sum").fillna(0).astype(int))
```

- **Line 32.** The per-recording rows as a table.
- **Line 33.** Saves it.
- **Line 34.** A heading.
- **Line 35.** A `wrong` column, the opposite of `correct` (~ flips true and false).
- **Line 36.** Counts the wrong predictions for each word and target: `pivot_table` puts words in rows and targets in columns and sums the trues. Words with no errors get 0.

Expected output with your data. valence (handcrafted + LogReg): true non-positive 14 correct, 4 predicted positive; true positive 4 predicted non-positive, 8 correct. Recall 0.78 and 0.67, macro-F1 0.72. Arousal (CNN + LogReg): true high 4 correct, 6 missed; true low-mod 6 predicted high, 14 correct. Recall 0.40 and 0.70, macro-F1 0.55.

19 Is the accuracy really above the baseline?

Code cell 20

Notebook heading: Is the accuracy really above the baseline?

A model that beats the baseline by a few people may just have been lucky with these 30. This cell asks two questions for each target's best model: what range of accuracies is consistent with the result (the 95% Wilson interval), and how likely a score this high would be if the model were really only as good as the baseline (a one-sided binomial test).

Uses: `df`, `preds`, `results`, `top_model`.
Creates: `sig` (one row per target).
Saves to the results folder: `accuracy_significance.csv`.

```
1 sig = []
2 for target in TARGETS:
3     y = df[target].values
4     best = top_model(target)
```

```
5     p = preds[(target, int(best["cfg_id"]))]
6     n, correct = len(y), int((p == y).sum())
7     baseline = results[target].query("model == 'Baseline'").iloc[0]["accuracy"]
```

- **Line 1.** An empty list for the results.
- **Line 2.** For valence and for arousal:
- **Line 3.** The true labels.
- **Line 4.** The highest-scoring configuration.
- **Line 5.** Its predictions.
- **Line 6.** `n` = 30 and `correct` = the number of matching predictions (comparing two arrays gives true/false for each person; summing counts the trues).
- **Line 7.** The baseline's accuracy, read from the baseline row of the results table made in code cell 18 (0.600 for valence, 0.667 for arousal).

```
8     test = binomtest(correct, n, p=baseline, alternative="greater")
9     ci = binomtest(correct, n).proportion_ci(confidence_level=0.95, method="wilson")
```

- **Line 8.** The one-sided binomial test. It treats each of the 30 predictions as a trial that is correct with probability equal to the baseline accuracy, and computes the probability of getting `correct` or more right. `alternative="greater"` makes it one-sided, because the question is only whether the model is better.
- **Line 9.** The 95% Wilson score interval for the model's accuracy. The interval does not depend on the baseline, so it is computed from a second `binomtest` call without one.

Formula. binomial p-value with k correct out of n and baseline accuracy p_0: $p = P(X \geq k),\ X \sim \text{Binomial}(n, p_0)$. Wilson interval with $\hat{p} = k/n$ and $z = 1.96$:

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$$

```
10    sig.append({"target": target, "best": f"{best['features']} ({best['model']})",
11                "correct": f"{correct}/{n}", "accuracy": round(correct / n, 3),
12                "wilson_95_low": round(ci.low, 3), "wilson_95_high": round(ci.high, 3),
13                "baseline_accuracy": baseline, "baseline_inside_CI": ci.low <= baseline
      ↪ <= ci.high,
14                "binomial_p_one_sided": round(test.pvalue, 3)})
```

- **Lines 10–14.** One result row: target, configuration, "22/30"-style count, accuracy, the interval's lower and upper ends, the baseline accuracy, whether the baseline lies inside the interval (`ci.low <= baseline <= ci.high` is true or false), and the p-value.

```
15 sig = pd.DataFrame(sig)
16 sig.to_csv(out_dir / "accuracy_significance.csv", index=False)
17 display(sig)
```

- **Lines 15–17.** The rows as a table, saved and shown.

Expected output with your data. valence 22/30 = 0.733, interval 0.556–0.858, baseline 0.600 inside, p = 0.094. Arousal 18/30 = 0.600, interval 0.423–0.754, baseline 0.667 inside, p = 0.835. With 30 people, valence would have needed 23 correct for $p < 0.05$.

20 Sensitivity check without the two Libog recordings

Code cell 21

Notebook heading: Without the two Libog recordings

Only two participants said "Libog", the eighth word the group added. This cell reruns each target's highest-scoring configuration on the 28 other recordings, with the same evaluation, to show how much the results depend on those two people.

Uses: `df`, `best_cfg`, `feature_sets`, `results`, `make_model`.
Creates: `df2` (28 rows), `folds2`, `no_libog_results`.
Saves to the results folder: `sensitivity_without_libog.csv`.

```
1  no_libog = (df["word"] != "Libog").values
2  df2 = df[no_libog].reset_index(drop=True)
3  folds2 = list(LeaveOneGroupOut().split(df2, groups=df2["participant"]))
```

- **Line 1.** True for every recording whose word is not Libog.
- **Line 2.** The table without those two rows, renumbered 0–27.
- **Line 3.** New leave-one-participant-out folds for the 28 participants (`df2` is passed only so the function knows how many rows there are).

```
5  rows = []
6  for target in TARGETS:
7      config = best_cfg[target]
8      X2 = feature_sets[config["features"]][no_libog]
9      y2 = df2[target].values
10     p = cross_val_predict(make_model(config["model"]), X2, y2, cv=folds2)
11     base = cross_val_predict(DummyClassifier(strategy="most_frequent"),
       ↪ np.zeros((len(y2), 1)), y2, cv=folds2)
12     correct, base_acc = int((p == y2).sum()), accuracy_score(y2, base)
13     full = results[target].query("features == @config['features'] and model ==
       ↪ @config['model']").iloc[0]
```

- **Line 5.** An empty list for the result rows.
- **Line 6.** For valence and for arousal:
- **Line 7.** The configuration chosen in code cell 19.
- **Line 8.** Its feature rows for the 28 recordings (a true/false array selects rows).
- **Line 9.** Their labels.
- **Line 10.** Leave-one-participant-out predictions with the same pipeline.

- **Line 11.** The baseline on the same 28 recordings, because the class sizes change.
- **Line 12.** The number correct and the new baseline accuracy.
- **Line 13.** The same configuration's row in the full 30-recording results. Inside `query`, `@config[...]` refers to the Python variable `config`.

```
14      rows.append({"target": target, "configuration": f"{config['features']}
        ↪ ({config['model']})",
15                   "all 30: accuracy": full["accuracy"], "all 30: macro-F1":
        ↪ full["macro_f1"],
16                   "without Libog: correct": f"{correct}/{len(y2)}", "without Libog:
        ↪ accuracy": round(correct / len(y2), 3),
17                   "without Libog: macro-F1": round(f1_score(y2, p, average="macro"), 3),
18                   "without Libog: baseline accuracy": round(base_acc, 3),
19                   "without Libog: binomial p": round(binomtest(correct, len(y2), base_acc,
        ↪ alternative="greater").pvalue, 3)})
```

- **Lines 14–19.** One row comparing all 30 with the 28: accuracy and macro-F1 with all 30; correct, accuracy, macro-F1, baseline accuracy and one-sided binomial p-value without Libog.

```
20  no_libog_results = pd.DataFrame(rows)
21  no_libog_results.to_csv(out_dir / "sensitivity_without_libog.csv", index=False)
22  display(no_libog_results.T)
```

- **Lines 20–21.** The rows as a table, saved.
- **Line 22.** Shown transposed (`.T`), so each measure is a row and the two targets are columns, which is easier to read.

Expected output with your data. valence 19/28 = 0.679 (macro-F1 0.668) against a 0.607 baseline, p = 0.284. Arousal 18/28 = 0.643, exactly the baseline. The conclusions do not change, but the valence margin shrinks.

21 Permutation test

Code cell 22

Notebook heading: Permutation test

The permutation test asks: if the audio had no connection at all to the labels, how often would this model score as well as it did? It shuffles the labels 1000 times (which destroys any real connection but keeps 18/12 and 20/10), reruns the whole leave-one-participant-out evaluation each time, and compares the real macro-F1 with the 1000 shuffled ones.

Uses: `df`, `best_cfg`, `results`, `top_model`, `run_lopo`, `N_PERM`, `SEED`.
Creates: `perm` (one row per target).
Saves to the results folder: `permutation_<target>.png` for both targets and `permutation_tests.csv`.

```
1  rng = np.random.default_rng(SEED)
2  perm = []
```

```
3  for target in TARGETS:
4      config = best_cfg[target]
5      y = df[target].values
6      observed = top_model(target)["macro_f1"]
7      baseline_f1 = results[target].query("model == 'Baseline'").iloc[0]["macro_f1"]
```

- **Line 1.** A random generator seeded with 42, so the 1000 shuffles are the same in every run.
- **Line 2.** An empty list for the results.
- **Line 3.** For valence and for arousal:
- **Line 4.** The configuration chosen in code cell 19.
- **Line 5.** The true labels.
- **Line 6.** The real macro-F1 of that configuration.
- **Line 7.** The baseline's macro-F1, stored for comparison.

```
9      shuffled_scores = []
10     for _ in range(N_PERM):
11         y_shuffled = rng.permutation(y)
12         shuffled_scores.append(f1_score(y_shuffled, run_lopo(config, y_shuffled),
       ↪ average="macro"))
13     shuffled_scores = np.array(shuffled_scores)
14     p_value = (np.sum(shuffled_scores >= observed) + 1) / (N_PERM + 1)
```

- **Line 9.** An empty list for the shuffled scores.
- **Line 10.** Repeats 1000 times:
- **Line 11.** Shuffles the 30 labels into a random order.
- **Line 12.** Runs the full 30-fold evaluation with the shuffled labels and records the macro-F1.
- **Line 13.** Turns the list into an array.
- **Line 14.** The p-value: the number of shuffled scores at least as high as the real one, plus 1, divided by 1000 + 1. The "+1" counts the real result as one of the possible outcomes, so the p-value can never be exactly 0.

Formula.

$$p = \frac{1 + \sum_{b=1}^{B} \mathbf{1}\left[F_1^{(b)} \geq F_1^{\text{obs}}\right]}{B + 1}, \qquad B = 1000$$

```
16     perm.append({"target": target, "best": f"{config['features']} ({config['model']})",
17                  "observed_macro_f1": observed, "baseline_macro_f1": baseline_f1,
18                  "chance_mean": round(shuffled_scores.mean(), 3), "n_perm": N_PERM,
       ↪ "p_value": round(p_value, 3)})
```

- **Lines 16–18.** One result row: target, configuration, real macro-F1, baseline macro-F1, the average shuffled score ("chance mean", the real chance level for macro-F1), the number of shuffles, and the p-value.

```
20      fig = plt.figure(figsize=(5, 3))
21      plt.hist(shuffled_scores, bins=20, color="lightgray")
22      plt.axvline(observed, color="crimson", label=f"observed = {observed}")
23      plt.title(f"{target} (p = {p_value:.3f}, B = {N_PERM})", fontsize=9)
24      plt.xlabel("Macro-F1, shuffled labels")
25      plt.legend()
26      plt.tight_layout()
27      save_fig(fig, f"permutation_{target}")
28      plt.show()
```

- **Line 20.** A new figure.
- **Line 21.** A histogram of the 1000 shuffled scores.
- **Line 22.** A red vertical line at the real score.
- **Line 23.** The title, with the p-value and the number of shuffles.
- **Lines 24–25.** The x-axis label and the legend.
- **Lines 26–28.** Spacing, save, show.

```
30  perm = pd.DataFrame(perm)
31  perm.to_csv(out_dir / "permutation_tests.csv", index=False)
32  perm
```

- **Lines 30–31.** The rows as a table, saved.
- **Line 32.** A table name alone on the last line of a cell is displayed automatically.

Expected output with your data. valence 0.722 against a chance mean of 0.490, p = 0.018. Arousal 0.550 against 0.484, p = 0.300.

Note. This cell trains 1000 × 30 × 2 = 60,000 models and takes about 10 minutes. It does not account for having picked the best of six configurations first: the shuffles only re-run the chosen one. The baseline's macro-F1 (0.375) is lower than the chance mean because the baseline never predicts the smaller class, whose F1 is therefore 0.

22 Same word, different feeling

Code cells 23, 24

Notebook heading: Same word, different feeling

Kapoy alone is 13 of the 30 recordings, and the words are linked to the feelings (only 4 of the 13 Kapoy speakers rated themselves positive). A model could partly learn which word was said instead of how it was said. These two cells test that. Code cell 23 compares the classes within each word; code cell 24 removes the word differences from the features and reruns the evaluation.

Uses: `df`, `feat_df`, `results`, `folds`, `make_model`.

Creates: `lex`, `per_word`, `both`, `X_centred`, `lex_results`.

Saves to the results folder: `lexically_controlled_descriptives.csv`, `lexically_controlled_results.csv`.

Code cell 23

```
1 lex_feats = ["f0_mean", "f0_std", "rms_mean", "rms_max", "zcr_mean", "duration"]
2 lex = pd.concat([df[["word", "valence_class"]], feat_df[lex_feats]], axis=1)
```

- **Line 1.** Six features to compare.
- **Line 2.** A table with the word, the valence class and those six features.

```
4 per_word = lex.groupby(["word", "valence_class"]).size().unstack(fill_value=0)
5 display(per_word)
6 both = per_word[(per_word > 0).sum(axis=1) >= 2].index.tolist()
7 print("words with both valence classes:", both)
```

- **Line 4.** Counts the recordings of each word in each valence class; `unstack` turns the classes into columns, with 0 where a word has none in a class.
- **Line 5.** Shows the counts.
- **Line 6.** The words that have at least one recording in both classes: counting the non-zero columns per row must give 2.
- **Line 7.** Prints them.

```
9  tables = []
10 for w in both:
11     t = lex[lex["word"] == w].groupby("valence_class").mean(numeric_only=True).round(3)
12     print("\n" + w)
13     display(t)
14     tables.append(t.assign(word=w).reset_index())
15 if tables:
16     pd.concat(tables).to_csv(out_dir / "lexically_controlled_descriptives.csv",
       ↪ index=False)
```

- **Line 9.** An empty list for the tables.
- **Line 10.** For each such word:
- **Line 11.** The average of each feature per valence class, for that word only.
- **Lines 12–13.** Prints the word and shows the table.
- **Line 14.** Keeps the table with the word added as a column.
- **Lines 15–16.** If any word qualified, joins the tables and saves them.

Code cell 24

```
1 X_centred = (feat_df - feat_df.groupby(df["word"].values).transform("mean")).values
```

- **Line 1.** Word-centring: `groupby(word).transform("mean")` gives, for every recording, the average of each feature over all recordings of the same word. Subtracting it leaves only how this recording differs from the typical recording of its word. Afterwards every word has an average of 0 on every feature, so the model cannot use differences between words.

Formula. for recording r with word $w(r)$ and feature j: $\tilde{x}_{rj} = x_{rj} - \bar{x}_{w(r),j}$, where $\bar{x}_{w,j}$ is the mean of feature j over all recordings of word w.

```
2  rows = []
3  for target in TARGETS:
4      y = df[target].values
5      p = cross_val_predict(make_model("LogReg"), X_centred, y, cv=folds)
6      base = cross_val_predict(DummyClassifier(strategy="most_frequent"), np.zeros((len(y),
       ↪ 1)), y, cv=folds)
7      original = results[target].query("features == 'Audio: handcrafted' and model ==
       ↪ 'LogReg'").iloc[0]["macro_f1"]
8      rows.append({"target": target,
9                   "uncentred handcrafted LogReg macro-F1": original,
10                  "word-centred macro-F1": round(f1_score(y, p, average="macro"), 3),
11                  "baseline macro-F1": round(f1_score(y, base, average="macro"), 3),
12                  "word-centred accuracy": round(accuracy_score(y, p), 3),
13                  "baseline accuracy": round(accuracy_score(y, base), 3)})
```

- **Line 2.** An empty list for the results.
- **Line 3.** For valence and for arousal:
- **Line 4.** The true labels.
- **Line 5.** Leave-one-participant-out predictions with the logistic-regression pipeline on the centred features.
- **Line 6.** The baseline predictions.
- **Line 7.** The macro-F1 of the same model on the original, uncentred features (from code cell 18), for comparison.
- **Lines 8–13.** One row: original macro-F1, word-centred macro-F1, baseline macro-F1, word-centred accuracy, baseline accuracy.

```
14 lex_results = pd.DataFrame(rows)
15 lex_results.to_csv(out_dir / "lexically_controlled_results.csv", index=False)
16 lex_results
```

- **Lines 14–15.** The rows as a table, saved.
- **Line 16.** Displayed automatically.

Expected output with your data. in the group report, valence macro-F1 fell from 0.72 to 0.58 (close to the 0.49 chance mean) and accuracy to 0.60, the baseline. Rerun the cell and check your numbers before quoting them.

Note. The word averages are computed from all 30 recordings, including the person being tested in each fold, so the test recording slightly affects its own centred values. Words with only 2 or 3 speakers make this effect larger. Treat the result as exploratory, as the notebook says.

23 Prototype

Code cell 25

Notebook heading: Prototype

The prototype takes any WAV file and predicts its valence and arousal class. It uses the handcrafted features and logistic regression, trained on all 30 recordings. It is a demonstration that the pipeline works from start to finish; how reliable it is comes from the leave-one-participant-out results, not from this cell.

Uses: `X_hand`, `df`, `feat_df`, `make_model`, `load_audio`, `get_features`, `TARGETS`.
Creates: `final` (one trained model per target), the function `predict_affect`, `out`.

```
1  final = {t: make_model("LogReg").fit(X_hand, df[t].values) for t in TARGETS}
```

- **Line 1.** Trains one logistic-regression pipeline per target on all 30 recordings, with no one left out. The result is a dictionary from target name to trained model.

```
4  def predict_affect(wav_path):
5      x = pd.DataFrame([get_features(load_audio(wav_path))])[feat_df.columns].fillna(0.0).
       ↪ values
6      return {t: m.predict(x)[0] for t, m in final.items()}
```

- **Line 4.** `predict_affect` takes the path of a WAV file.
- **Line 5.** Preprocesses it, computes the 67 features, puts them in a one-row table with the columns in the same order as during training, replaces any missing value with 0, and takes the plain numbers.
- **Line 6.** Asks each trained model for its prediction and returns, for example, `{"valence_class": "positive", "arousal_class": "high"}`.

```
9  for t, m in final.items():
10     right = int((m.predict(X_hand) == df[t].values).sum())
11     print(f"{t}: {right}/30 right on its own training data (baseline
       ↪ {df[t].value_counts().max()}/30)")
```

- **Line 9.** For each target and its model:
- **Line 10.** How many of the 30 recordings the model gets right when it predicts the same recordings it was trained on.
- **Line 11.** Prints that count next to the baseline count (the size of the larger class).

Expected output with your data. valence 22/30 (baseline 18/30); arousal 20/30 (baseline 20/30). Even on its own training data, the arousal model does no better than always guessing "low-mod".

```
13 example = audio_dir / "Y1-027_Y1_PM_Lingaw.wav"
14 pred = predict_affect(example)
15 r = df[df["file"] == example.name].iloc[0]
```

```
16 out = pd.DataFrame({"prediction": [pred[t] for t in TARGETS],
17                     "self-reported": [r[t] for t in TARGETS],
18                     "rating (1-5)": [r["valence"], r["arousal"]]}, index=TARGETS)
19 out["match"] = out["prediction"] == out["self-reported"]
20 print("\n" + example.name)
21 display(out)
```

- **Line 13.** The example file.
- **Line 14.** Its prediction.
- **Line 15.** That participant's row in the table, for the self-reported ratings.
- **Lines 16–18.** A small table with one row per target: the prediction, the self-reported class, and the 1–5 rating.
- **Line 19.** Whether prediction and self-report match.
- **Lines 20–21.** Prints the filename and shows the table.

Expected output with your data. Y1-027 "Lingaw": valence predicted positive, self-reported 5 (positive), match. Arousal predicted high, self-reported 2 (low-mod), no match.

Note. Y1-027 was one of the 30 training recordings, so this is not a test on new data. The wrong arousal prediction is consistent with the main result: no arousal model beat the baseline.

A Output files

Everything below is saved in `RESULTSV3/` (subfolders as shown).

File	Code cell	What it contains
`metadata_issues.csv`	5	Problems found in the metadata (only written if there are any)
`file_correspondence_check.csv`	6	One row per participant: WAV and MP4 found, ratings, classes
`exploratory_rating_summary.csv`	7	Summary statistics of the two ratings
`exploratory_rating_distribution.png`	7	Bar charts of the ratings 1–5
`exploratory_<column>_summary.csv`	7	Ratings by year level, activity, session, word
`exploratory_<column>.png`	7	Strip plots by activity, session, word
`exploratory_word_by_activity_counts.csv`	7	Word × activity counts
`rq1_self_reported_feelings_counts.csv`	8	Counts of the free-text feelings
`rq1_valence_class_by_arousal_class_counts.csv`	8	2 × 2 class table
`rq1_valence_arousal_scatter.png`	8	Jittered scatter of the ratings
`rq1_valence_by_arousal_rating_counts.csv`	8	5 × 5 rating counts
`rq1_valence_arousal_counts.png`	8	Heatmap of the rating counts
`audio_quality_report.csv`	9	Sample rate, channels, length, level, clipping, flags
`SPECTROGRAMS/*.png`	10	30 plain 224 × 224 images, the CNN input
`acoustic_features.csv`	10	The 67 features per recording
`spectrogram_examples.png`	12	Figure 1: 8 labelled spectrograms
`spectrograms_labelled/*.png`	12	30 labelled spectrograms
`waveforms_spectrograms/*.png`	13	30 waveform + spectrogram + pitch figures
`word_gallery_<word>.png`	13	All recordings of one word, sorted by valence
`rq3_<target>_acoustic_comparison.csv`	14	Medians, means, Mann–Whitney p-values
`rq3_<target>_acoustic_boxplots.png`	14	Boxplots of six features by class
`audio_descriptive_statistics.csv`	15	Summary of all 67 features
`audio_descriptive_by_<target>.csv`	15	Means and SDs of 12 features by class
`cnn_embeddings.npy`	16	30 × 512 ResNet18 values
`datasets/Group1_audio_feature_dataset.csv`	17	The final 30 × 78 dataset
`datasets/data_dictionary.csv`	17	Group of every dataset column
`model_results.csv`	18	Accuracy and macro-F1 of all configurations
`confusion_matrices.png`	19	Confusion matrices of the top models

File	Code cell	What it contains
`best_model_predictions.csv`	19	Every recording's prediction from the top models
`accuracy_significance.csv`	20	Wilson interval and binomial test
`sensitivity_without_libog.csv`	21	Results without the two Libog recordings
`permutation_<target>.png`	22	Histograms of the shuffled scores
`permutation_tests.csv`	22	Permutation p-values and chance means
`lexically_controlled_descriptives.csv`	23	Feature averages per class within each word
`lexically_controlled_results.csv`	24	Word-centred results

B Variables reference

The main variables, in the order they are created. When a later cell fails with a `NameError`, the variable it names was created in the cell listed here, which has not been run yet.

Variable	Cell	What it holds
`SEED`	2	The random seed, 42
`audio_dir`, `video_dir`, `out_dir`	3	Input and output folders
`SR`, `CLIP_SEC`, `N_MELS`, `IMG_SIZE`	3	Audio settings: 16 kHz, 1.5 s, 128 bands, 224 px
`N_PCA`, `N_PERM`	3	5 PCA components, 1000 shuffles
`WORDS`	3	The eight allowed response words
`meta`	4	The metadata CSV exactly as read
`df`	4	Working table, one row per recording, with `participant`, `word`, `path`, `valence_class`, `arousal_class`
`meta_by_file`	4	The metadata indexed by audio filename
`problems`, `gaps`	5	Metadata issues; missing participant numbers
`files`	6	File-matching table
`quality`	9	Audio quality table
`clips`	10	Processed audio per recording (row number → samples)
`img_paths`	10	Paths of the 30 CNN images
`feat_df`, `X_hand`	10	The 67 features as a table and as an array (30×67)
`vec`	11	The 67 features of the demo recording
`X_cnn`	16	ResNet18 values (30×512)
`dataset`	17	The saved 30×78 dataset
`feature_sets`, `TARGETS`	18	The two feature sets; the two targets
`folds`	18	The 30 leave-one-participant-out folds

Variable	Cell	What it holds
`configs`	18	The 7 configurations (0 = baseline)
`results`	18	Score table per target, sorted by macro-F1
`preds`	18	Predictions per (target, configuration number)
`best_cfg`	19	The highest-scoring configuration per target
`pred_df`	19	Per-recording predictions of the top models
`sig`	20	Interval and binomial test per target
`perm`	22	Permutation results per target
`X_centred`	24	Word-centred features
`final`	25	The two trained prototype models

C Settings at a glance

Step	Setting	Where
Resampling	192 kHz stereo → 16 kHz mono	`load_audio`, cell 10
Trimming	25 dB below the peak, both ends	`load_audio`, cell 10
Normalisation	peak = 1.0	`load_audio`, cell 10
Silence for timing features	30 dB below the peak	`get_features`, cells 9, 10, 13
Frames for features	2048 samples (128 ms), hop 512 (32 ms)	librosa defaults, cell 10
Pitch range	65–500 Hz (pYIN)	cells 10, 13
MFCCs	13 coefficients; deltas over 5 frames	cell 10
CNN images	1.5 s, n_fft 1024, hop 128, 128 mel bands, 224 × 224	cell 10
Report figures	n_fft 512, hop 64 (4 ms), up to 8 kHz	cell 13
Class threshold	rating > 3 → positive / high	cell 4
Scaling + PCA	StandardScaler, 5 components	cell 18
Logistic regression	C = 0.5, balanced, max_iter 2000	cell 18
SVM	RBF kernel, C = 1.0, balanced	cell 18
Random forest	100 trees, depth 4, min leaf 2, balanced	cell 18
Evaluation	leave-one-participant-out, 30 folds	cell 18
Permutation test	1000 shuffles, seed 42	cell 22

D Glossary

Term	Meaning
Sample rate	How many numbers per second describe the sound. 16 kHz = 16,000 per second.
Frame, hop	A short piece of audio analysed at once, and the step to the next piece.
dB (decibel)	A logarithmic loudness scale; +6 dB roughly doubles the amplitude. Here usually relative to the clip's peak (0 dB = loudest).
Mel scale	A frequency scale spaced like human pitch perception: about even up to 1 kHz, compressed above.
Spectrogram	Strength of each frequency over time, drawn as an image.
MFCC	Numbers that summarise the shape of the spectrum of each frame.
f0 / pitch	The fundamental frequency of the voice; pYIN is the algorithm that estimates it.
Feature vector	The 67 numbers that describe one recording.
Embedding	The 512 numbers ResNet18 produces for one spectrogram image.
PCA	Combines many correlated features into a few uncorrelated components that keep most of the variation.
Pipeline	Steps (scaling, PCA, classifier) trained and applied together.
LOPO	Leave-one-participant-out: train on 29 people, test on the 30th, repeat for everyone.
Baseline	Always predicting the most common class; the minimum a useful model must beat.
Accuracy	Share of correct predictions.
Precision, recall	Of those predicted as a class, how many were right; of those truly in a class, how many were found.
F1, macro-F1	Combination of precision and recall per class; the average over both classes.
Confusion matrix	Counts of true class against predicted class.
p-value	How likely a result at least this good would be if there were no real effect. Lower is stronger evidence; 0.05 is the usual cut-off.
Wilson interval	A 95% range for the true accuracy, accurate for small samples.
Permutation test	Repeating the whole evaluation with shuffled labels to see what chance alone produces.